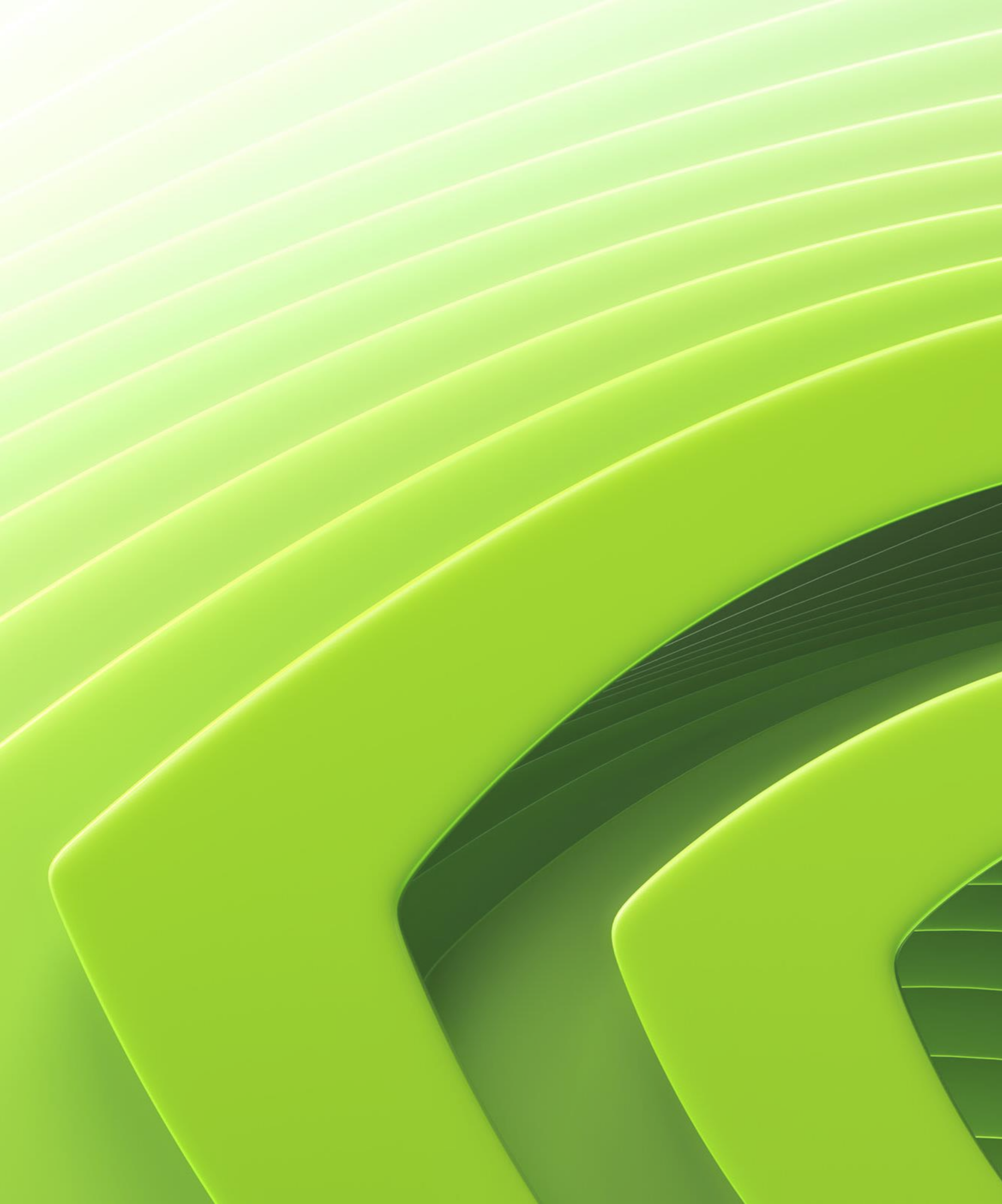




GPU Communication Libraries for Accelerating HPC and AI Applications

Benjamin Glick, Pouya Kousha, GPU Communications & Networking, NVIDIA

IEEE HOT Interconnects, 2026

- 
- 0. Introduction
 - 00. Hello world toolchain check
 - 1. NCCL host APIs
 - 2. NVSHMEM RMA
 - 2a. Host put on a CUDA stream
 - 2b. Device put
 - 2c. Direct peer mapping with `nvshmem_ptr`
 - 3. Memory semantics
 - 4. NCCL Symmetric Memory
 - 5. NCCL device APIs
 - 6. Python APIs
 - 7. NCCL contrib and extensions
 - 8. Applications
 - Questions & feedback

Motivation and Goals

Why GPU Communication Matters ?

- Modern AI and HPC workloads require multiple GPUs to work together efficiently
 - TOP500 graphs showing considerable share of multi-GPU
 - Fast, scalable GPU-to-GPU communication
- Technologies like NVLink, PCIe, and RDMA (InfiniBand, RoCE, etc) enable high-bandwidth, low-latency data transfer between GPUs

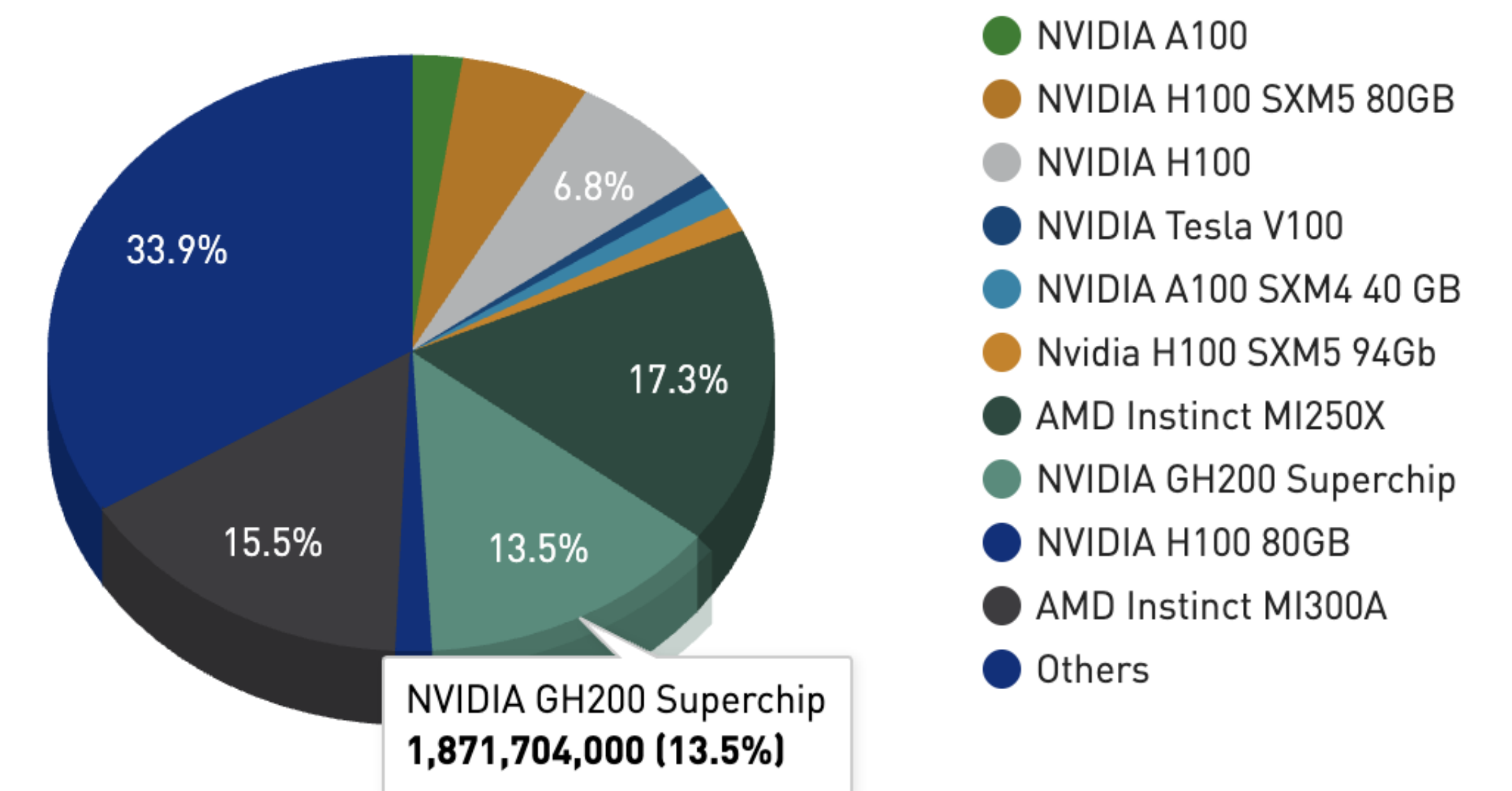
Common Use Cases

- Distributed deep learning (e.g., PyTorch, vLLM, DeepEP, TRTLLM, etc).
- Large-scale simulations and scientific computing.
- Real-time data analytics and inference pipeline

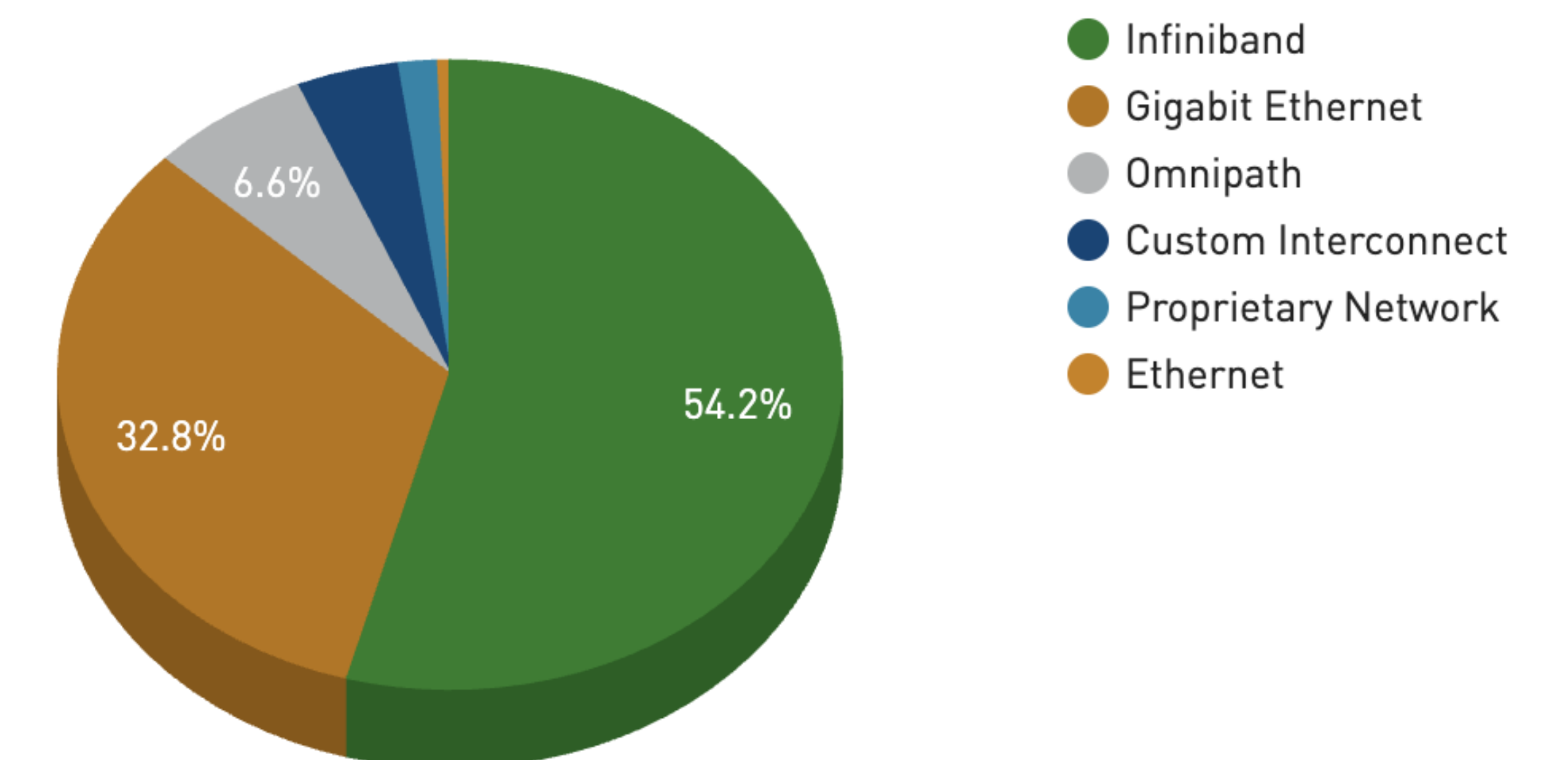
What will you learn ?

- Lean about Nvidia solutions for efficient data movement between GPUs
- **NCCL** for low-latency and high-throughput GPU-GPU communication
- **NVSHMEM** for fine-grained GPU-centric communication

Accelerator/Co-Processor Performance Share



Interconnect Family System Share



CUDA Streams

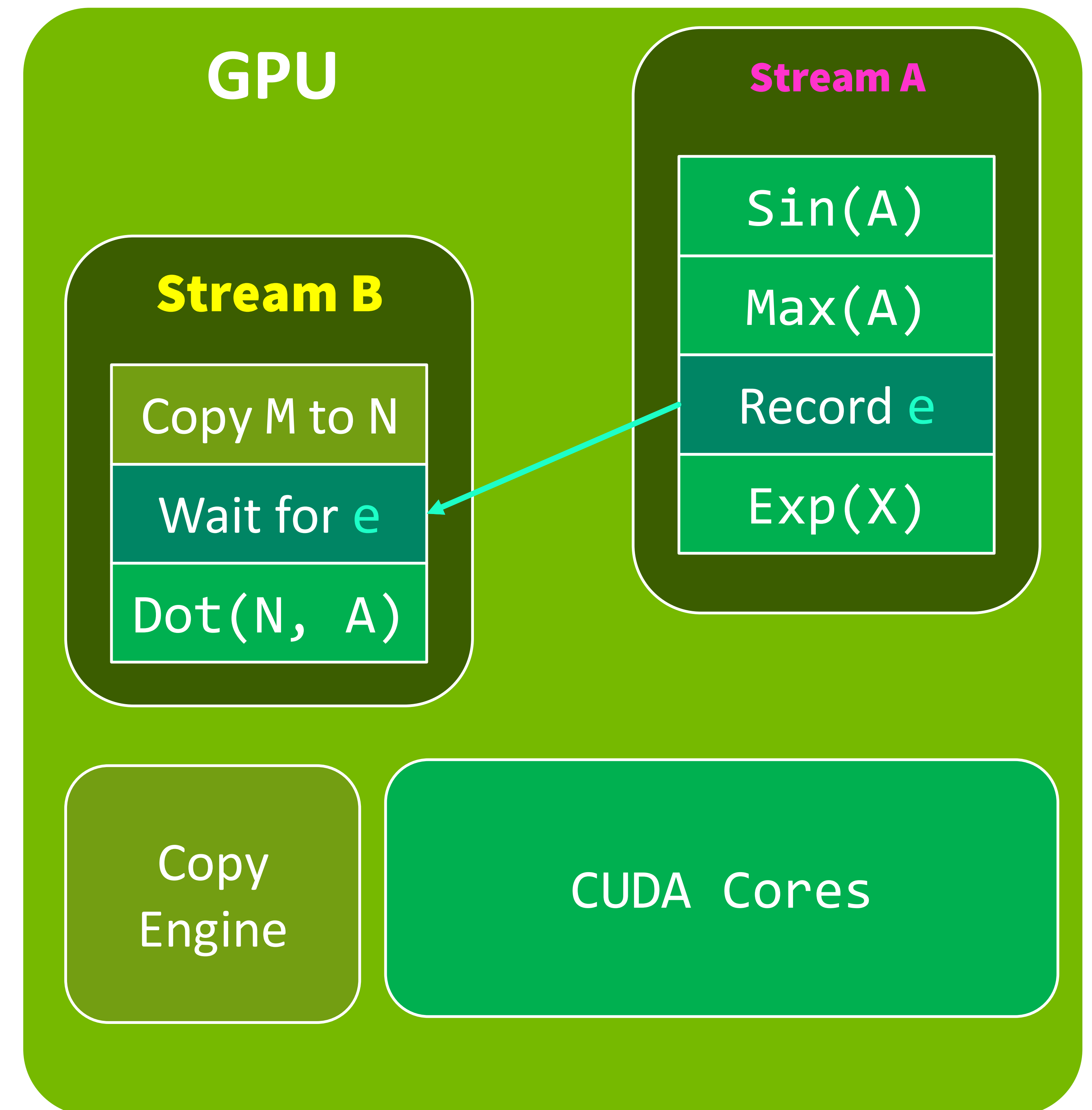
- A Stream is a work queue for GPU actions
- Streams run in order, and are allowed (not guaranteed) to run in parallel
- Events can be used to synchronize across streams

Create a stream in CUDA-Python with:

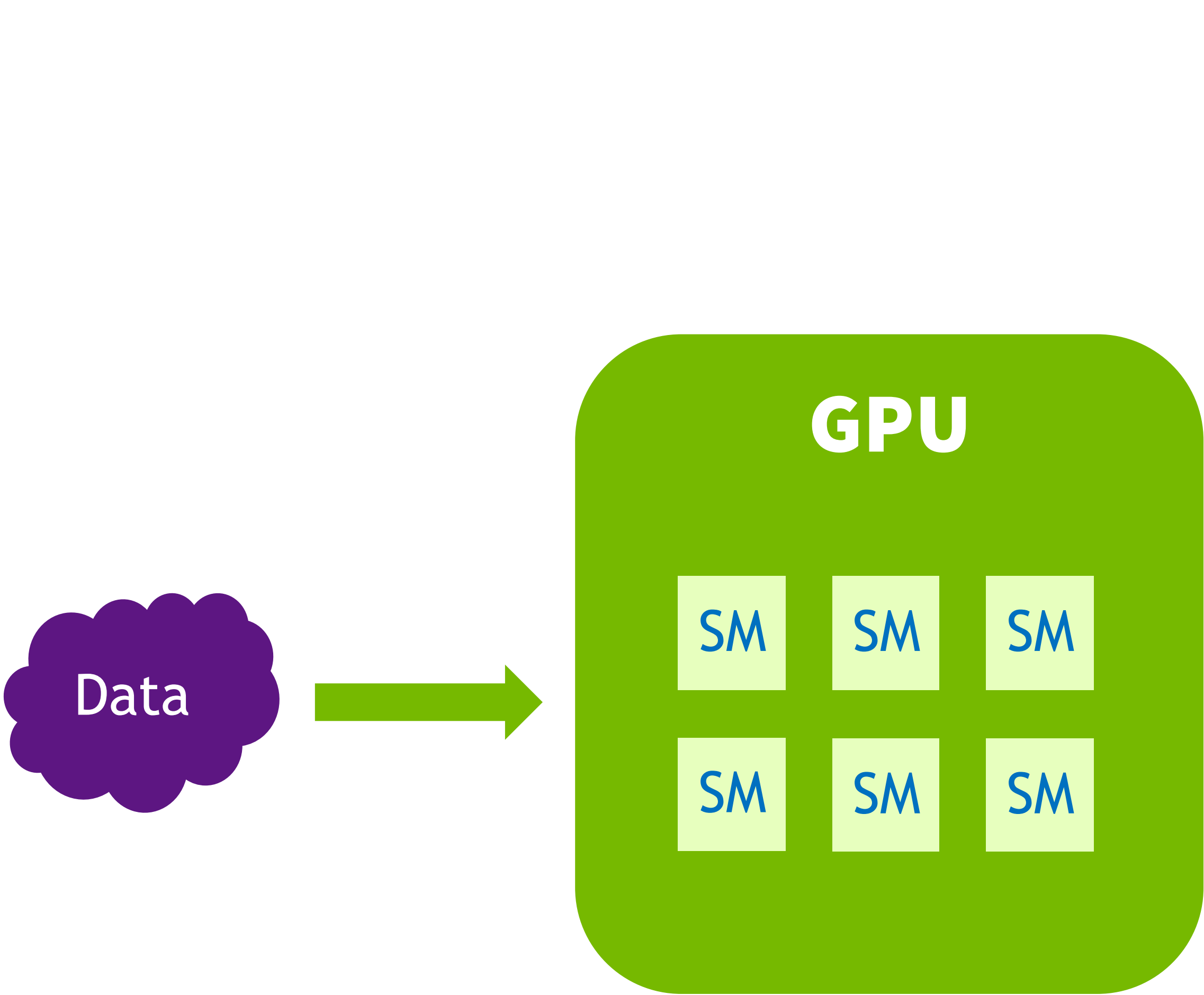
```
from cuda.core import Device
```

```
dev = Device()  
dev.set_current()
```

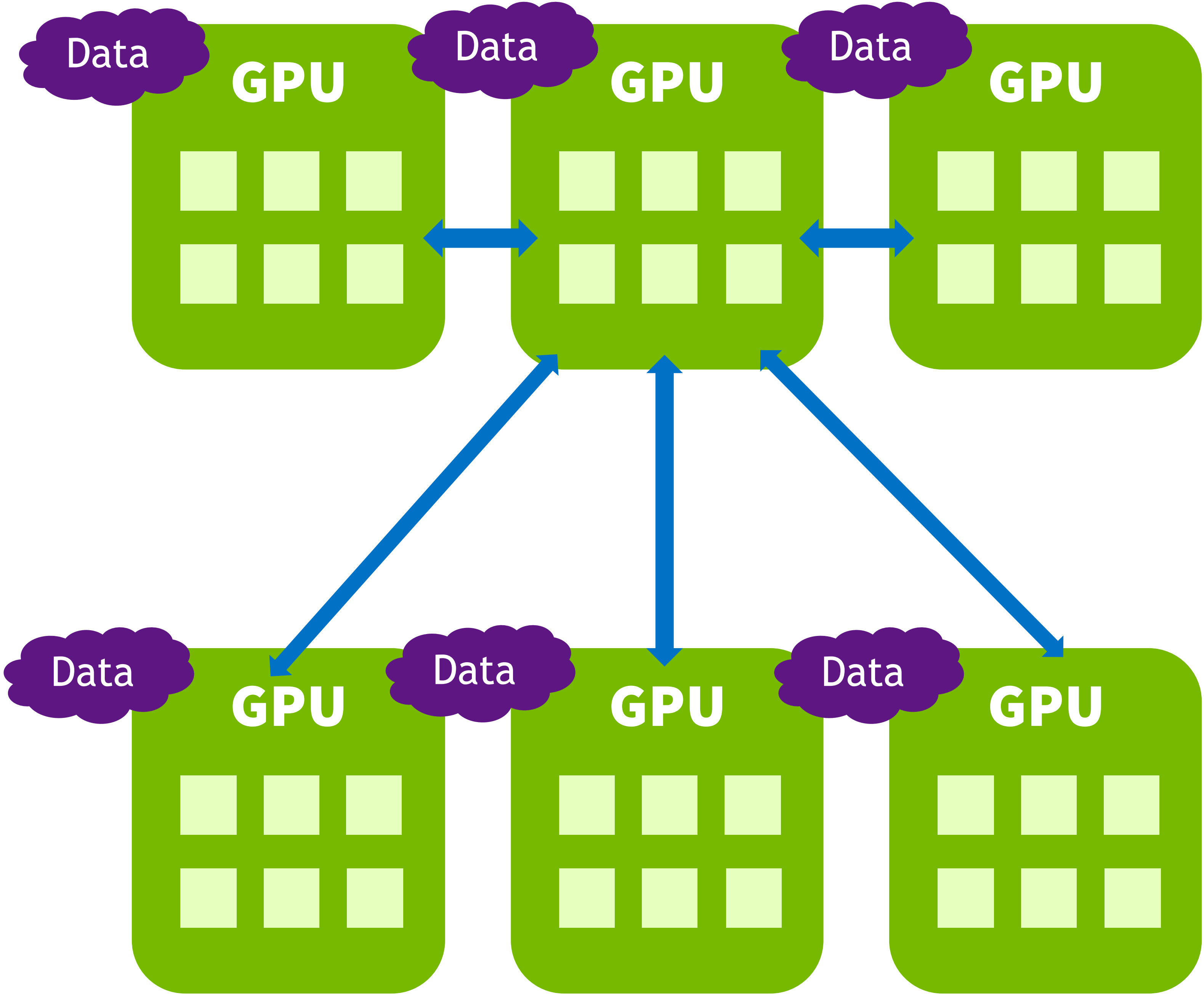
```
stream = dev.create_stream()
```



Single GPU -> Multi GPU



Single GPU



Multi GPU

Exercise 00: run the hello-world check

Build and run the completed source before the coding labs

No TODOs. If this fails, fix the CUDA or MPI environment before moving on.

Find us in Slack if you run into problems.

```
00-intro/00-hello-world/
```

```
make
```

```
make run
```

Default: two MPI ranks on visible GPUs 0 and 1.

Expected output (order does not matter):

```
hello from rank 0
```

```
hello from rank 1
```

Other nodes:

```
make run NP=1 CUDA_VISIBLE_DEVICES=2
```

```
make run NP=2 CUDA_VISIBLE_DEVICES=2,3
```

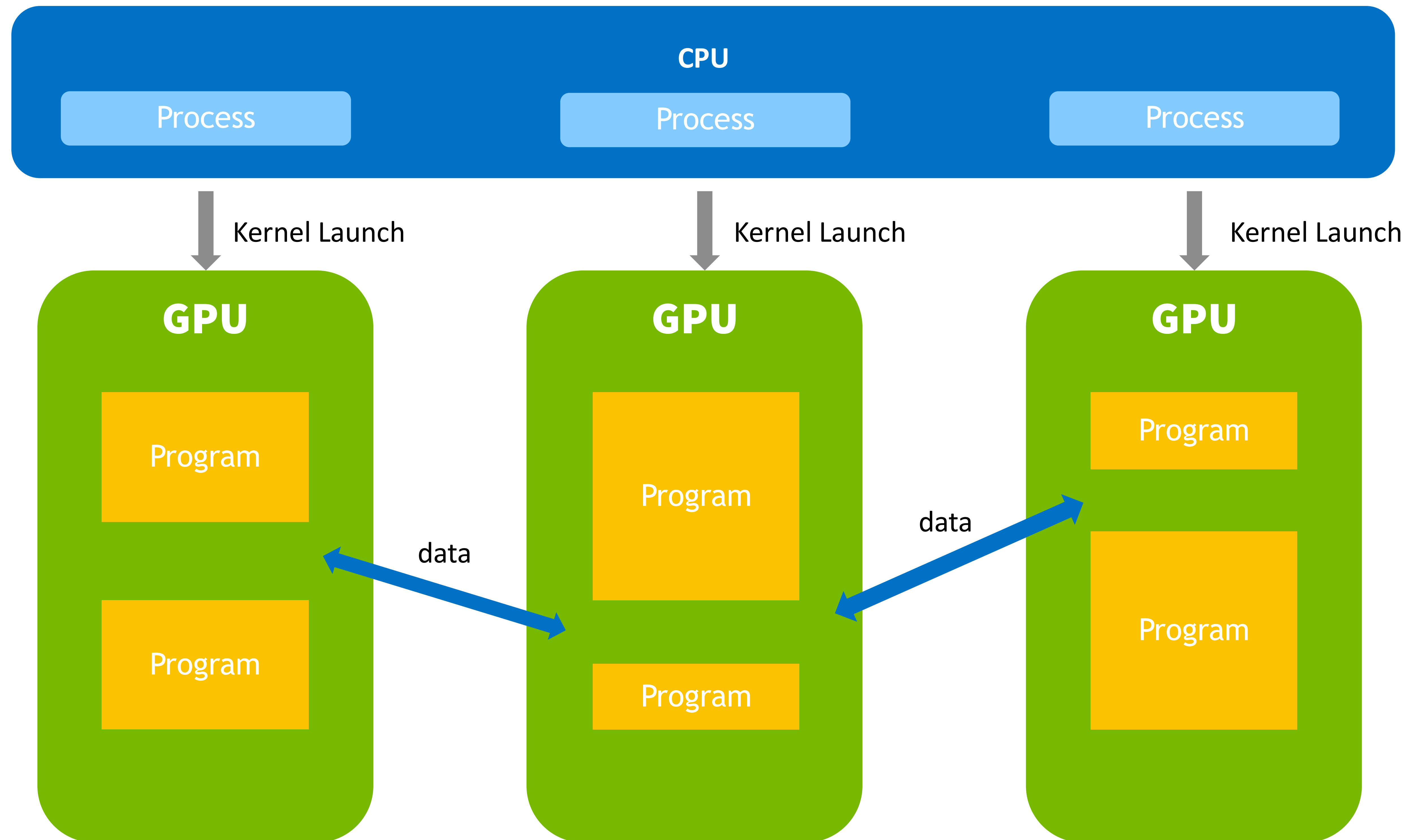


1. NCCL host APIs

Multi-GPU Programs

Workloads are launched across multiple GPUs and managed by multiple processes

When all processes participate in communication, it's called a *collective*

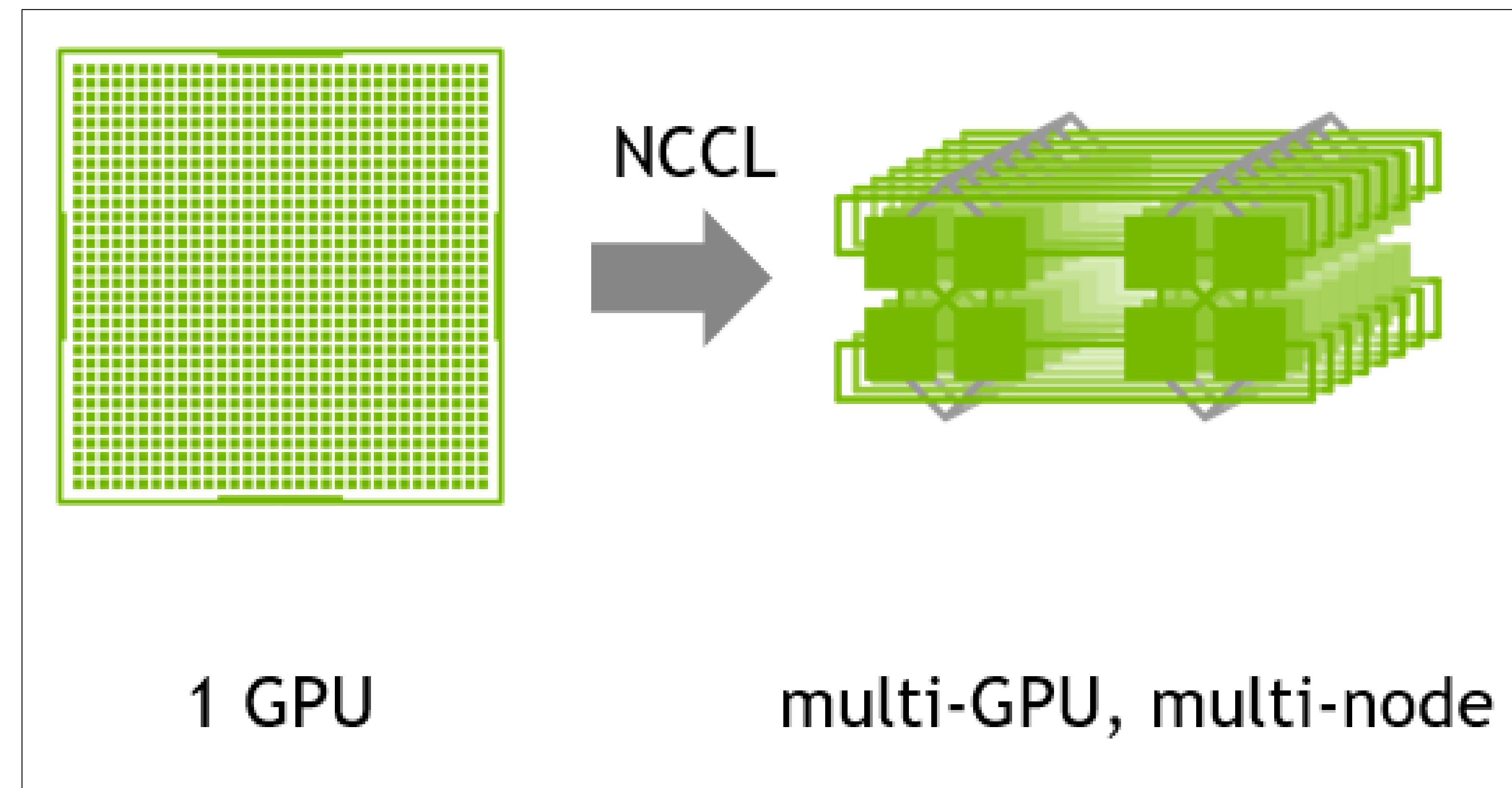


Optimized inter-GPU communication

NCCL : NVIDIA Collective Communication Library

Communication library running on GPUs, for GPU buffers.

- NCCL (pronounced “Nickel”) is a library developed by NVIDIA for efficient communication between multiple GPUs
 - Supports single node and across multiple nodes
- Collective and Pt to Pt Operations (e.g. Allreduce, Broadcast)
- **Host Library:** Communication calls are translated to a GPU kernel (running on a CUDA stream)
- **Device Library:** Primitives for nvlink and network communication initiated within GPU kernels



Binaries : <https://developer.nvidia.com/nccl> and in NGC containers

Source code : <https://github.com/nvidia/nccl>

Perf tests : <https://github.com/nvidia/nccl-tests>



NCCL host APIs

NCCL APIs

Communication

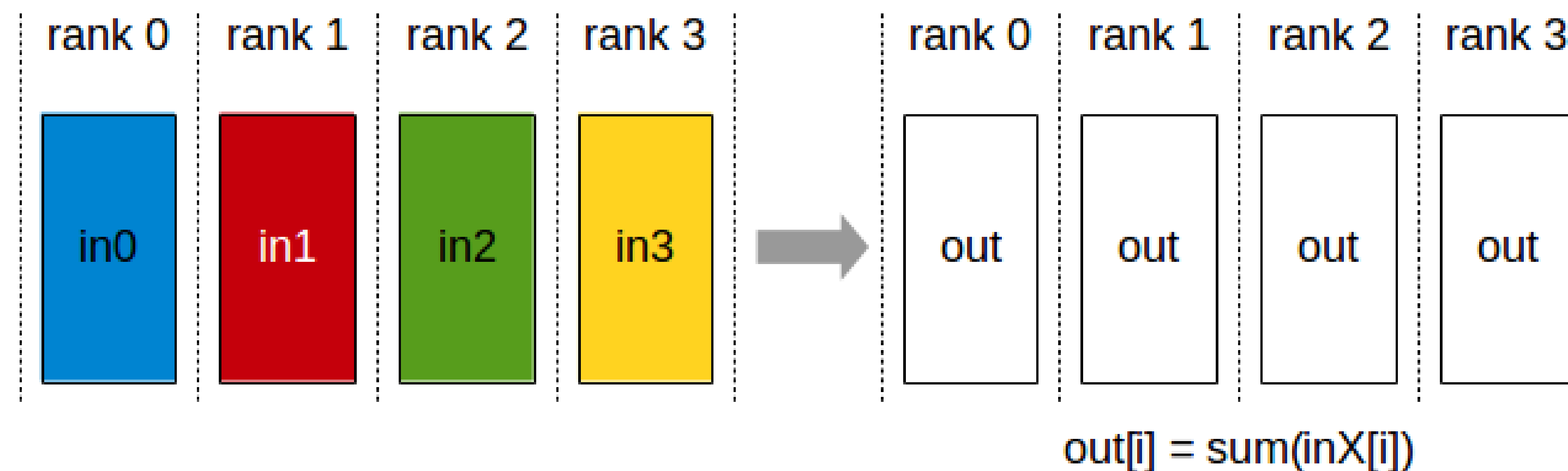
- Send/Recv

```
ncclSend(void* sbuff, size_t count, ncclDataType_t type, int peer, ncclComm_t comm, cudaStream_t stream);  
ncclRecv(void* rbuff, size_t count, ncclDataType_t type, int peer, ncclComm_t comm, cudaStream_t stream);
```

- Collective Operations

```
ncclAllReduce(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, ncclRedOp_t op,  
              ncclComm_t comm, cudaStream_t stream);  
ncclBroadcast(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, int root,  
              ncclComm_t comm, cudaStream_t stream);  
ncclReduce(void* sbuff, void* rbuff, size_t count, ncclDataType_t type, ncclRedOp_t op, int root,  
           ncclComm_t comm, cudaStream_t stream);
```

- Allreduce example



NCCL API

Send/Receive with NCCL

```
...  
// Initialized NCCL communicator  
  
int N=16;  
cudaStream_t stream;  
cudaStreamCreate(&stream);  
if (rank == 0) {  
    ncclSend(send_buf, N, ncclInt, 1, nccl_comm, stream);  
} else if (rank == 1) {  
    ncclRecv(recv_buf, N, ncclInt, 0, nccl_comm, stream);  
}  
  
cudaStreamSynchronize(stream);  
  
// Destroy NCCL communicator  
...
```

Creating GPU stream for NCCL stream

Sending 16 * ncclInt to rank 1 as part of nccl_comm

NCCL API

Fused Communication Calls

- Multiple calls to `ncc1Send()` and `ncc1Recv()` should be fused with `ncc1GroupStart()` and `ncc1GroupEnd()` to
Avoid deadlocks, e.g. if calls need to progress concurrently
For more performance: fused operations can be more efficient by better utilizing the available IO

Send/Recv

```
ncc1GroupStart();  
ncc1Send(sendbuff, sendcount, sendtype, peer, comm, stream);  
ncc1Recv(recvbuff, recvcount, recvtype, peer, comm, stream);  
ncc1GroupEnd();
```

Bcast:

```
ncc1GroupStart();  
if (rank == root) {  
    for (int r=0; r<n ranks; r++)  
        ncc1Send(sendbuff[r], size, type, r, comm, stream);  
}  
ncc1Recv(recvbuff, size, type, root, comm, stream);  
ncc1GroupEnd();
```

NCCL host APIs — Exercises 1a and 1b

Stream-ordered point-to-point and collective communication

1a Send/receive on a stream — `send_recv_on_stream.c`

1b All-reduce on a stream — `allreduce_on_stream.c`

[01-nccl-host-apis/](#)

1a SEND / RECV

```
ncclSend(sendbuf, count, datatype, peer, comm, stream);  
ncclRecv(recvbuf, count, datatype, peer, comm, stream);
```

Matched GPU-buffer transfer. Count and type must match; the local stream orders surrounding CUDA copies.

1b ALL-REDUCE

```
ncclAllReduce(sendbuf, recvbuf, count, datatype,  
              ncclSum, comm, stream);
```

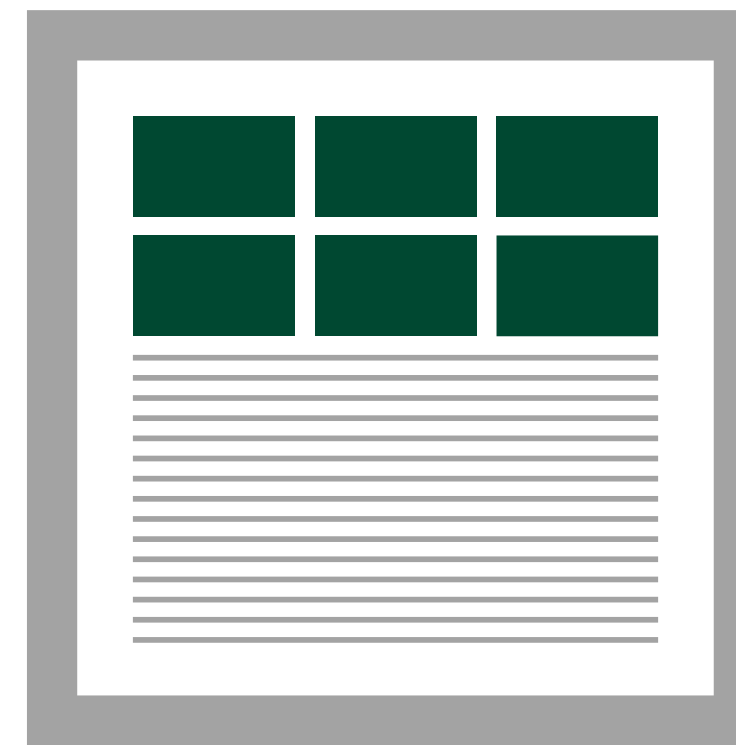
Every rank in the communicator calls it; every rank receives the element-wise sum.

2. NVSHMEM RMA

NVSHMEM

Overview

- Implements & Extends the OpenSHMEM API for clusters of NVIDIA GPUs
- Partitioned Global Address Space (PGAS) programming model
 - One sided Communication with put/get
 - Shared memory Heap
- GPU Centric communication APIs
 - GPU Initiated: thread, warp, block (narrow datatypes and tensor-operands)
 - CPU Initiated: Stream/Graph-Based (communication kernel or cudaMemcpyAsync)
- Since 3.3: First-party language bindings for Python Ecosystem
 - Will be covered in the new features section



NVSHMEM



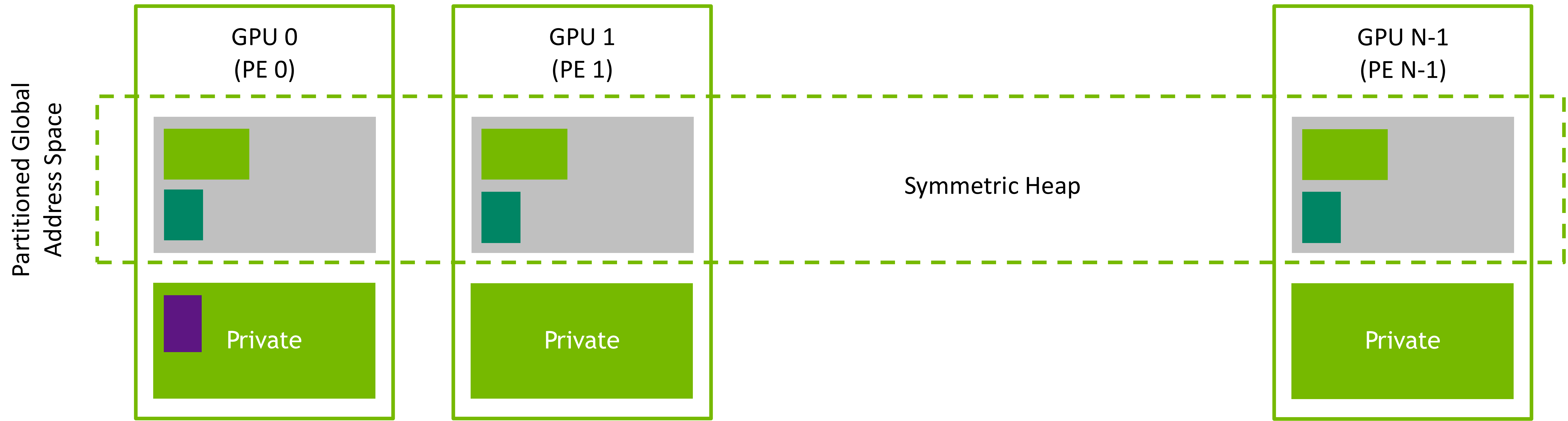
Project Home Page: <https://docs.nvidia.com/nvshmem/>

Developer Forum Page:

<https://forums.developer.nvidia.com/tag/nvshmem>

NVSHMEM

Memory Model



Symmetric objects are allocated collectively with the same size on every PE

- Symmetric memory: `nvshmem_malloc(size);`
- Private memory: `cudaMalloc(...)`

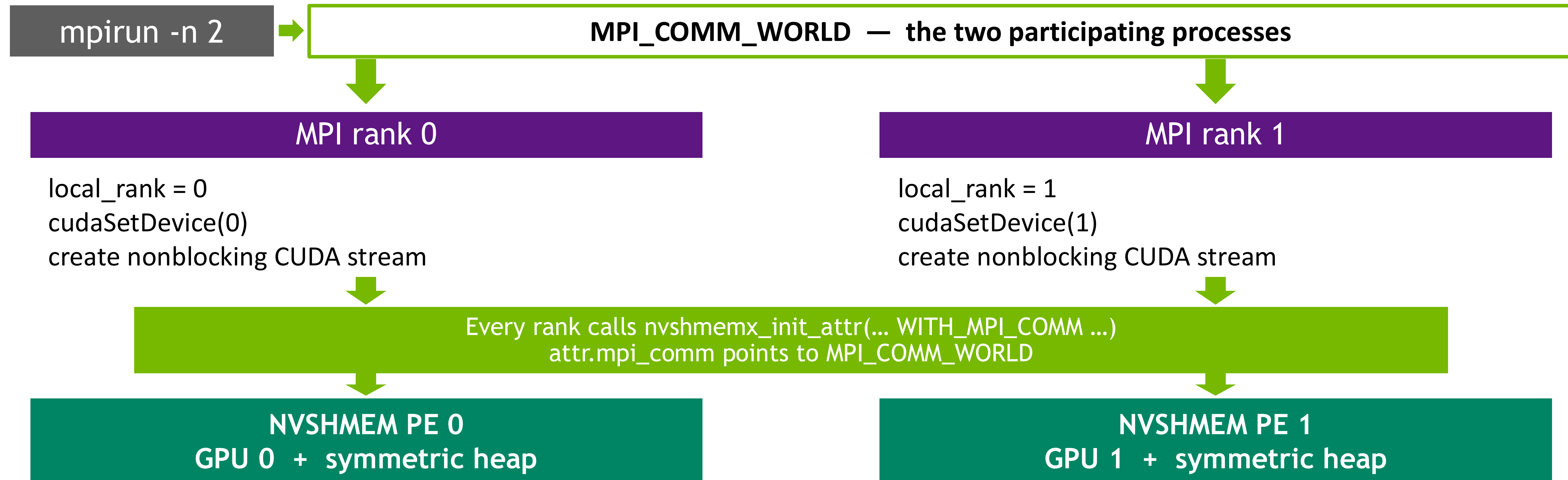
Must be the same
on all PEs



NVSHMEM APIs

NVSHMEM Initialization from MPI Comm

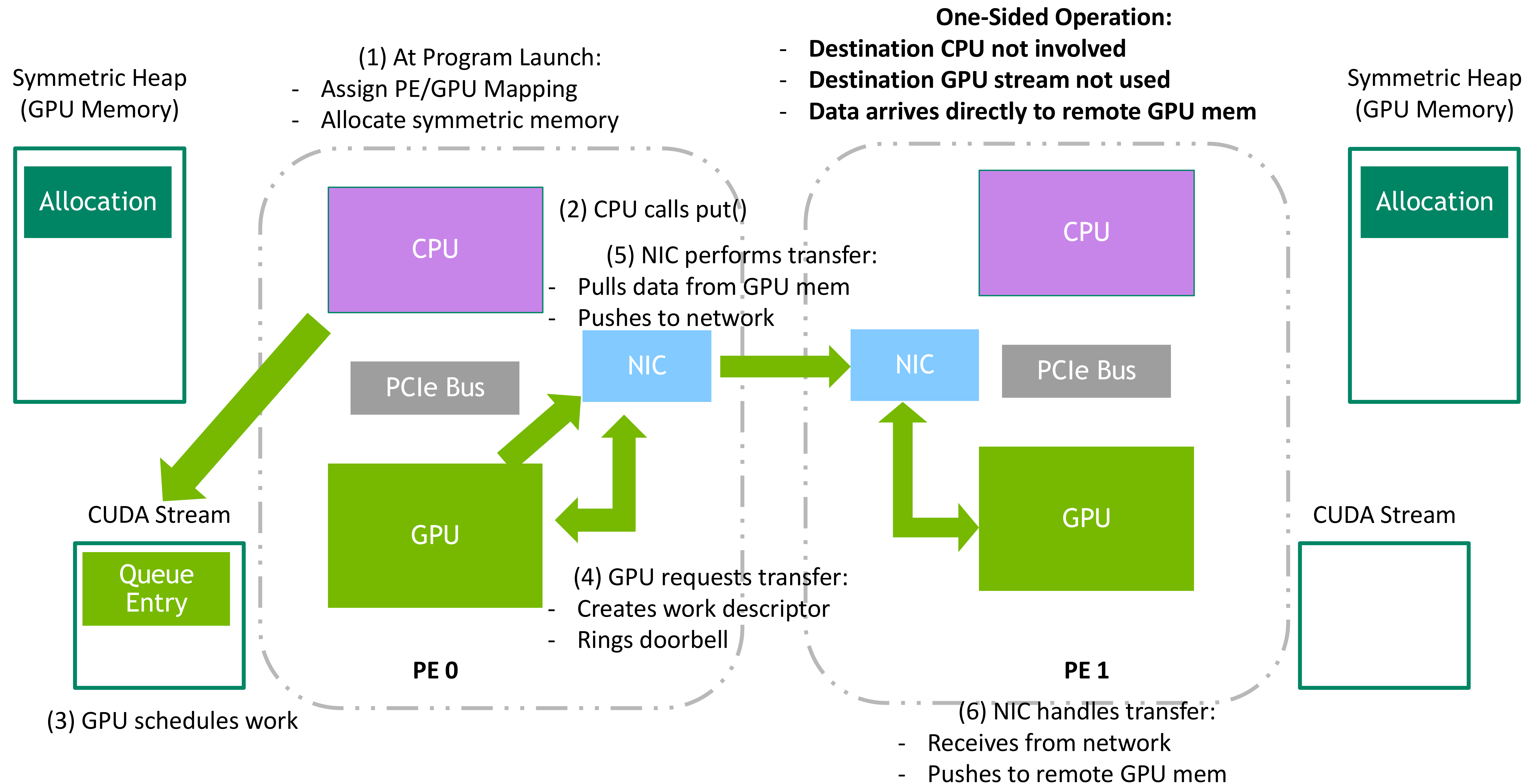
One MPI process becomes one NVSHMEM PE in this tutorial



GPU selection uses the node-local rank; the full MPI communicator defines the PE group.

```
MPI_Init(...);
attr.mpi_comm = &mpi_comm;
nvshmemx_init_attr(NVSHMEMX_INIT_WITH_MPI_COMM, &attr);
```

NVSHMEM Put Data Flow

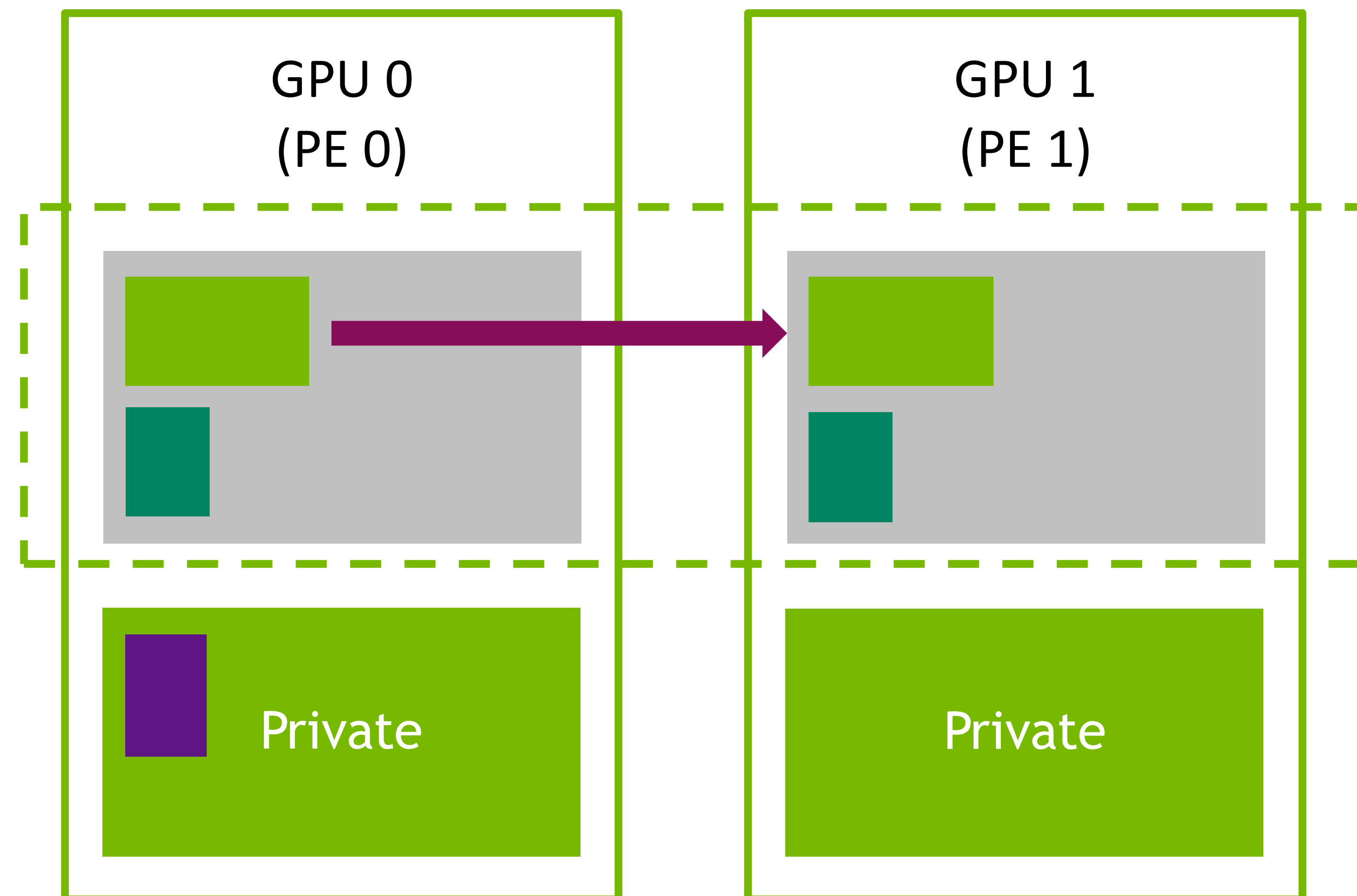


PE = "Processing Element"

Based on slides from Lena Oden @SC21, ISC22, SC22, ISC23: https://github.com/FZJ-JSC/tutorial-multi-gpu/blob/v4.0-isc23/07-L_Overview_of_NCCL_and_NVSHMEM_in_MPI_Programs/slides.pdf

2a. Host put on a CUDA stream

PE 0 queues one write into PE 1's symmetric target



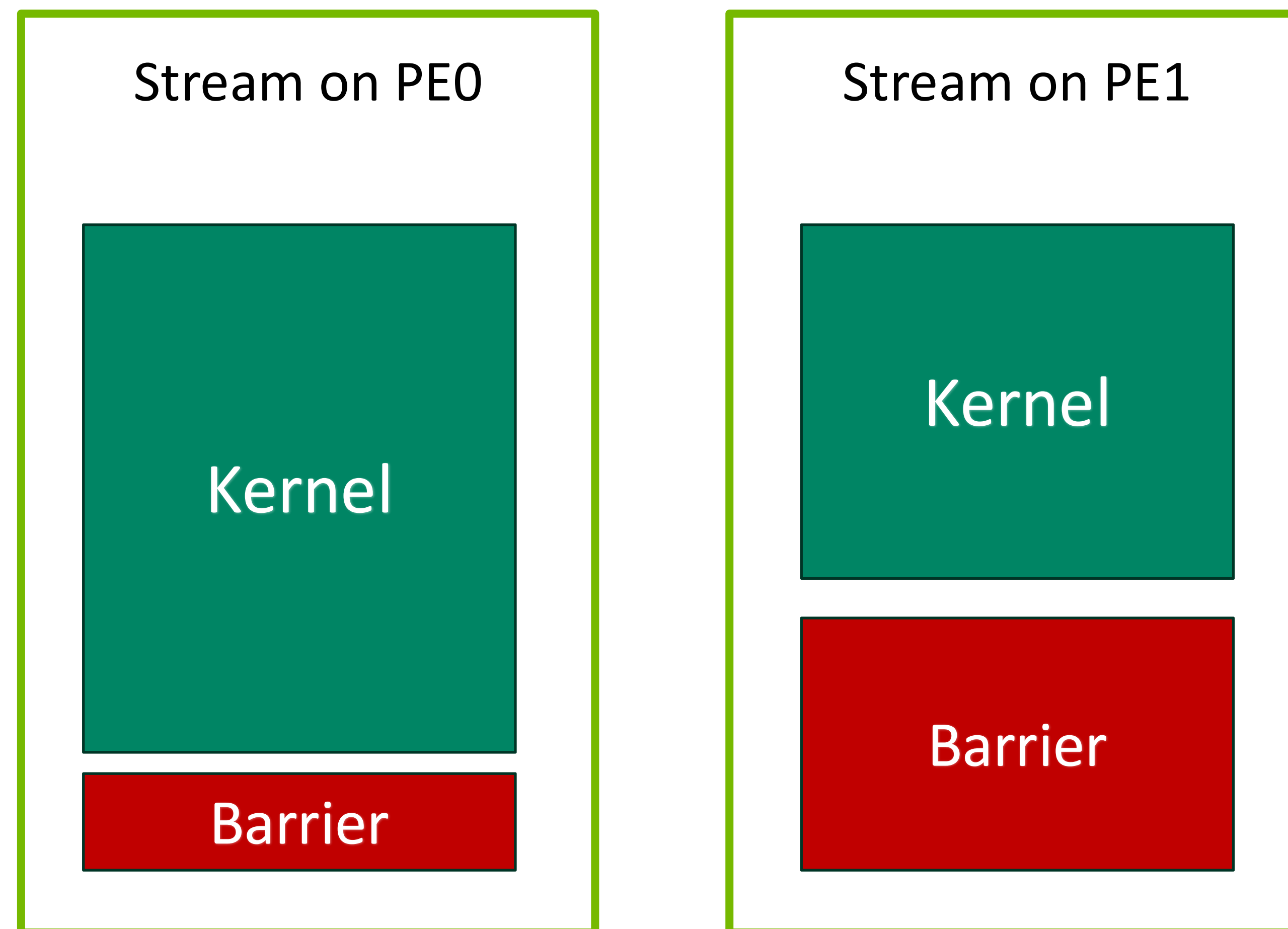
PE 1 does not post a receive. All work happens on sender side

```
if (rank == 0) {  
    nvshmemx_putmem_on_stream(target, source,  
                              sizeof(int), 1, stream); // Stream is a cudaStream_t  
}
```

Host queues RMA.

Host-side Barrier

Queue it after the put on every PE's stream



All PEs enter the barrier. It synchronizes the processes.

It also completes prior communication

```
nvshmemx_barrier_all_on_stream(stream); // There are also barriers on teams
cudaStreamSynchronize(stream);
// PE 1 may now read target[0]
```

Remember that the barrier occurs on a stream, asynchronously from the CPU's perspective!

Exercise 2a: host put on a stream

PE 0 sends 42 to PE 1; the barrier makes the result safe to read

Complete one host RMA put and one stream-ordered global barrier.

02-nvshmem-rma/01-host-put-on-stream/

```
nvshmemx_putmem_on_stream(void* dest, void* src, int bytes, int pe, cudaStream_t stream)
```

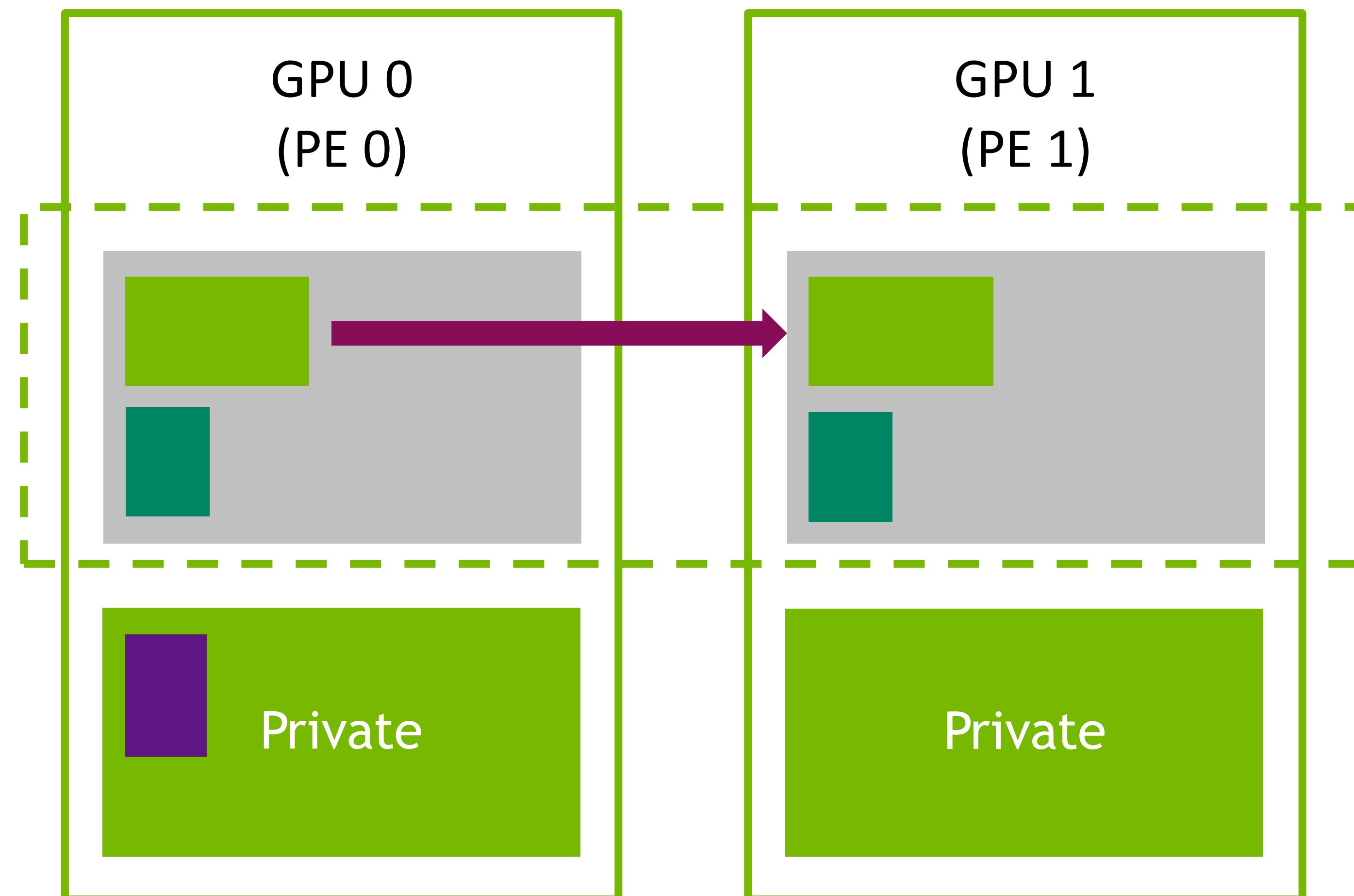
Queues a one-sided write to PE pe's symmetric destination.

```
nvshmemx_barrier_all_on_stream(stream)
```

Every PE enters the stream-ordered barrier; read after the local stream completes.

2b. Device put

One CUDA thread issues the RMA operation from the GPU



Operations like put can happen on the device or host side

Device operations run inside a kernel, which is run on a stream.

```
__global__ void put_kernel(int *dst, int *src, int nelems, int peer) {  
    nvshmem_int_put_[warp|block](dst, src, nelems, peer);  
    nvshmem_barrier_all();  
}
```

NVSHMEM device APIs
are scoped

Exercise 2b: device put

Keep the protocol simple; move the put into the kernel

Complete the device `nvshmem_int_put` call. Leave the barrier in place.

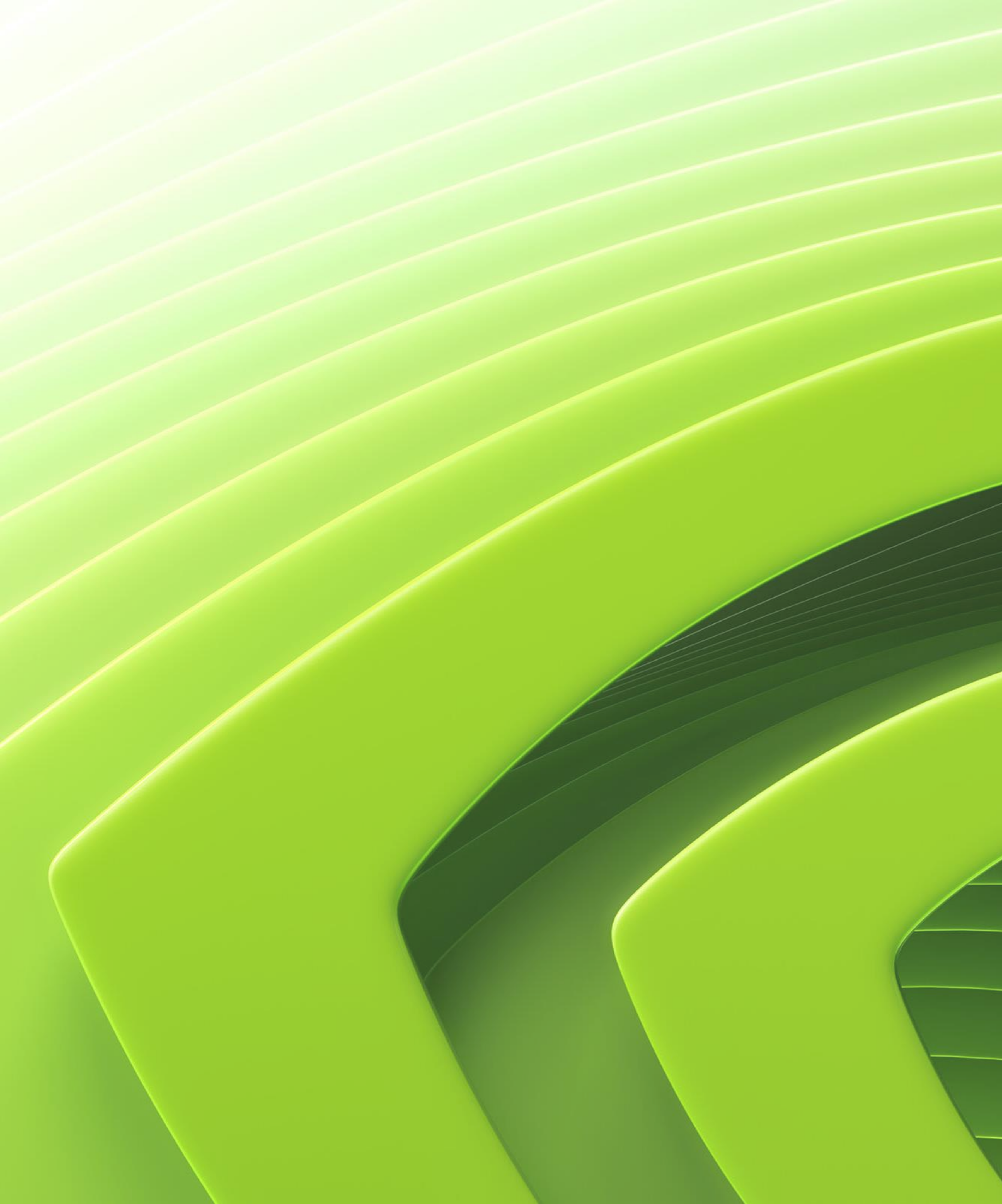
`02-nvshmem-rma/02-device-put/`

```
nvshmem_int_put(dest, source, nelems, pe)
```

Device-side put of `nelems` ints from `source` to PE `pe`'s matching symmetric destination.

```
nvshmemx_barrier_all_on_stream(stream)
```

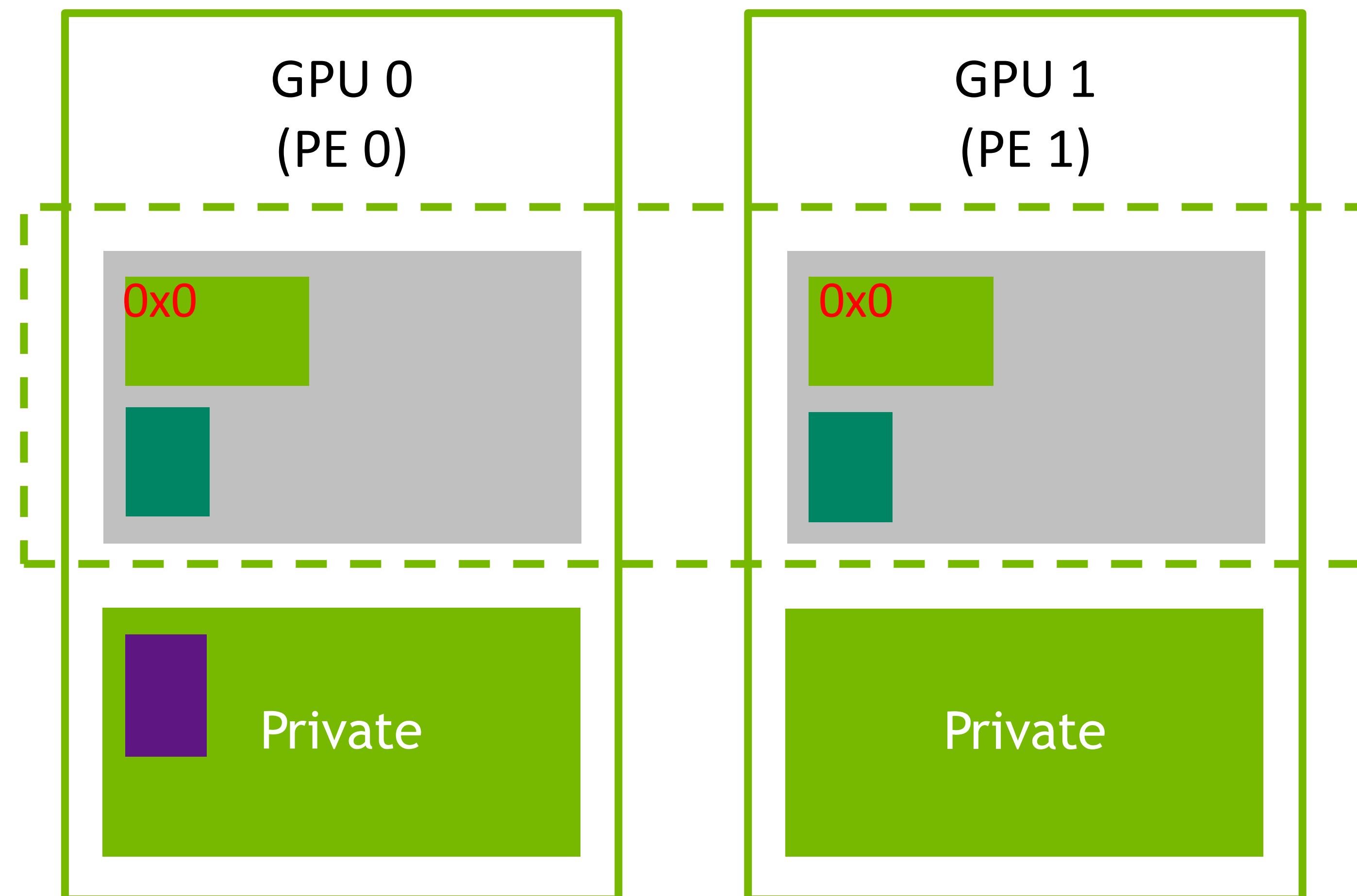
Queued after the kernel; provides the global handoff before the host reads.



2c. Direct peer mapping

2c. nvshmem_ptr

A direct peer mapping is not a synchronization primitive



If you want to perform irregular memory access or pointer arithmetic, you may use the `nvshmem_ptr(...)` function

The returned address is a peer mapping. You can dereference it

Remember symmetric addressing.

```
if (auto *peer_ptr = (int *)nvshmem_ptr(...)) {  
    *peer_ptr = value;  
}
```

Exercise 2c: nvshmem_ptr

Store through a peer mapping and make the store visible

Query the pointer, handle NULL, store through it, and follow the store with nvshmem_quiet()

02-nvshmem-rma/03-nvshmem_ptr/

```
void* nvshmem_ptr(const void* symmetric_ptr, int pe)
```

Returns a directly usable peer pointer, or NULL when that mapping is unavailable.

```
*peer_ptr = value; nvshmem_quiet()
```

Ordinary peer store; the quiet call orders it but does not notify the peer.



3. Memory semantics

Memory Model Brainteaser



```
int * val = nvshmem_malloc(sizeof(int));
*val = 42;

nvshmem_putmem_on_stream(val, val, 1, 1,
stream);
```

```
int * val = nvshmem_malloc(sizeof(int));

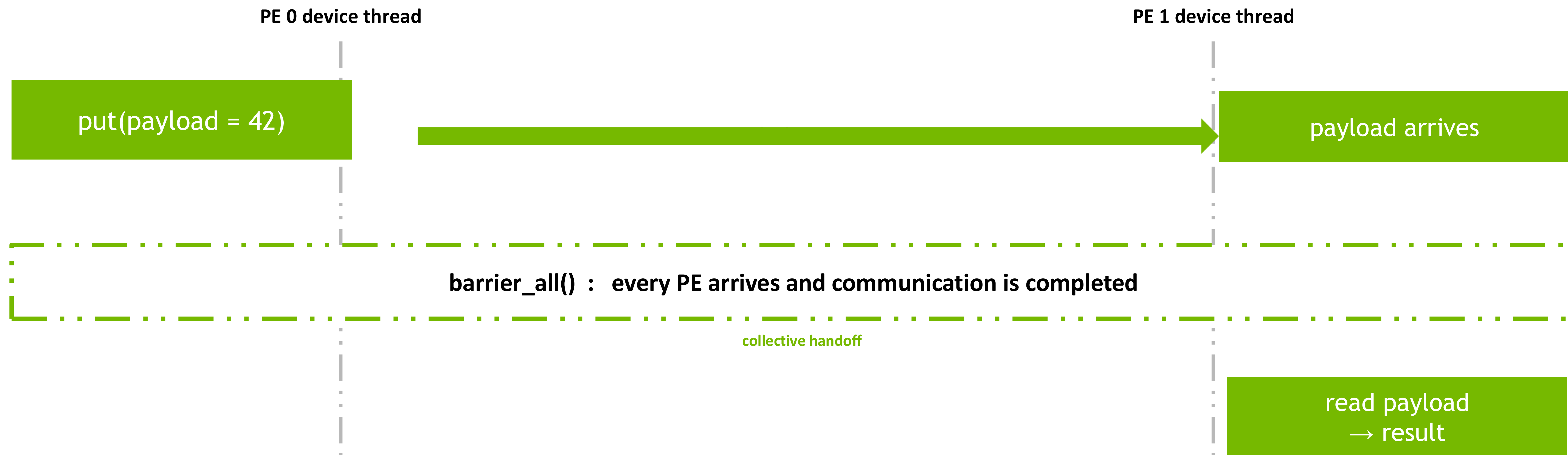
printf("Val on PE1: %d", *val);
```

What is val on PE0 and 1? Bonus points for “why”?

A: (42,42) B: (42, 0) C: (0, 42) D: Other

3a. Barrier

Both PEs participate before either one consumes payload



```
nvshmem_int_put(payload, source, 1, 1);    // PE 0
nvshmem_barrier_all();
```

NVSHMEM Quiet

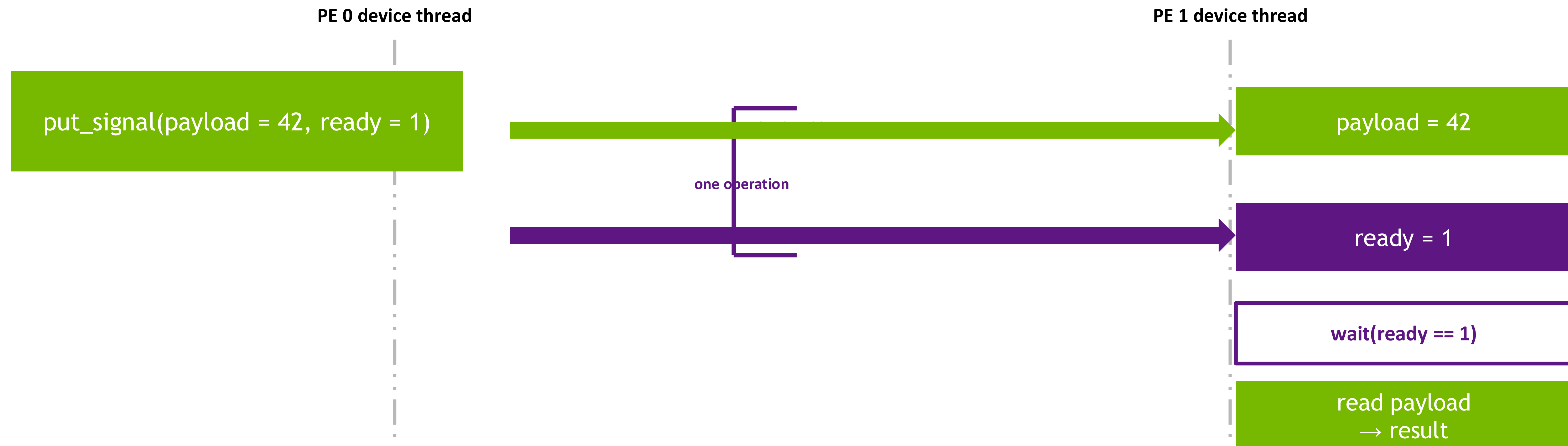
PE 0 completes its put, then separately notifies PE 1



```
nvshmem_int_put(payload, source, 1, 1);
nvshmem_quiet(); // only PE 0
nvshmemx_signal_op(ready, kReady, NVSHMEM_SIGNAL_SET, 1);
nvshmem_signal_wait_until(ready, NVSHMEM_CMP_EQ, kReady);
quiet completes PE 0's put; signal/wait is the peer handoff.
```

3c. put_signal couples data and notification

The signal becomes visible after the payload for this operation



```
nvshmem_int_put_signal(payload, source, 1, ready, kReady,
    NVSHMEM_SIGNAL_SET, 1);
nvshmem_signal_wait_until(ready, NVSHMEM_CMP_EQ, kReady);
// The put is ordered with respect to the signal. No other operations are.
```



4. Advanced NCCL features

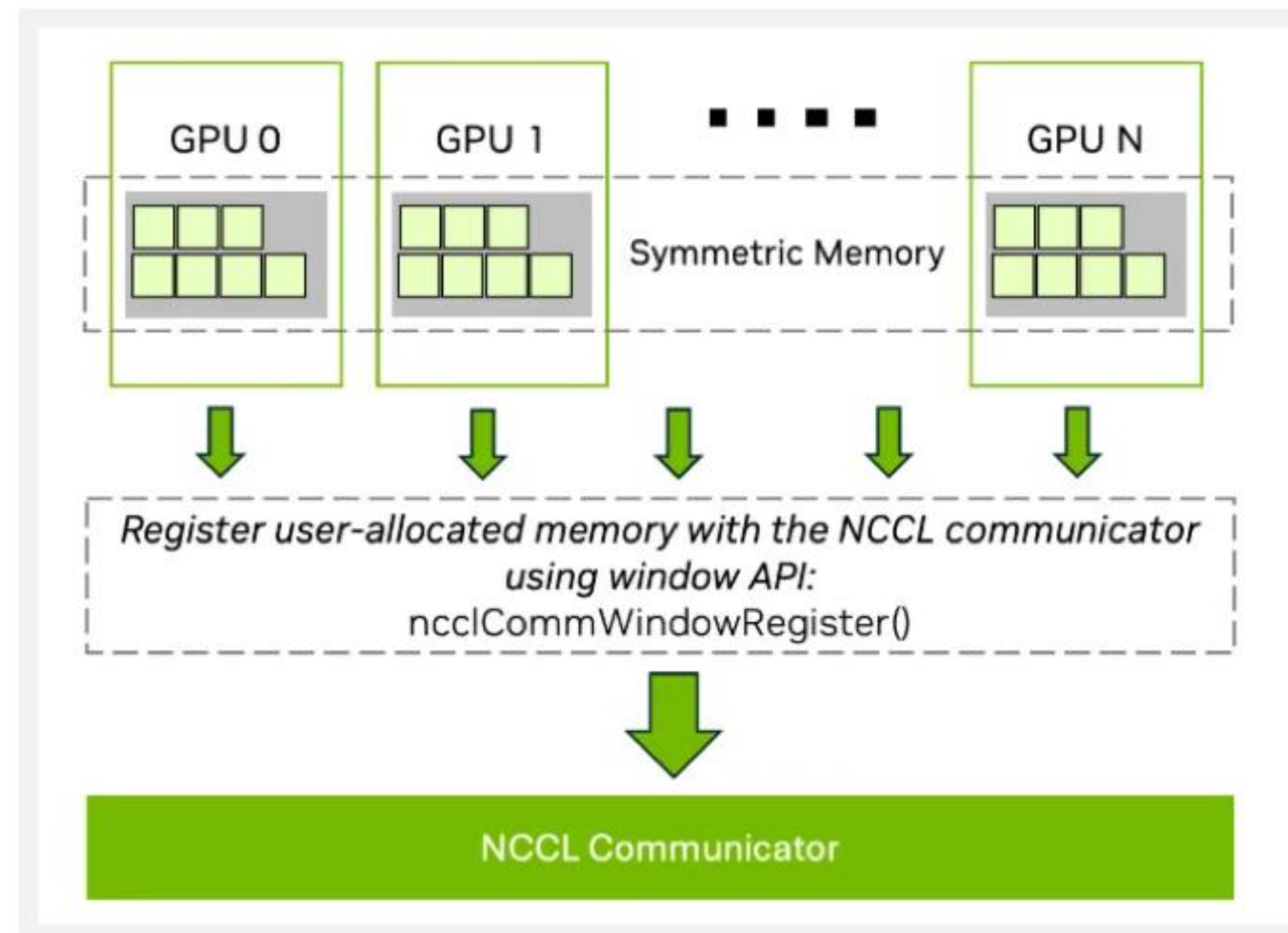
A symmetric window gives NCCL a common memory description

The layout is common across ranks even when the CUDA pointer values differ

Symmetric means that every rank registers the same logical layout. It does not mean the pointers or window handles are shared.

Without Symmetric:

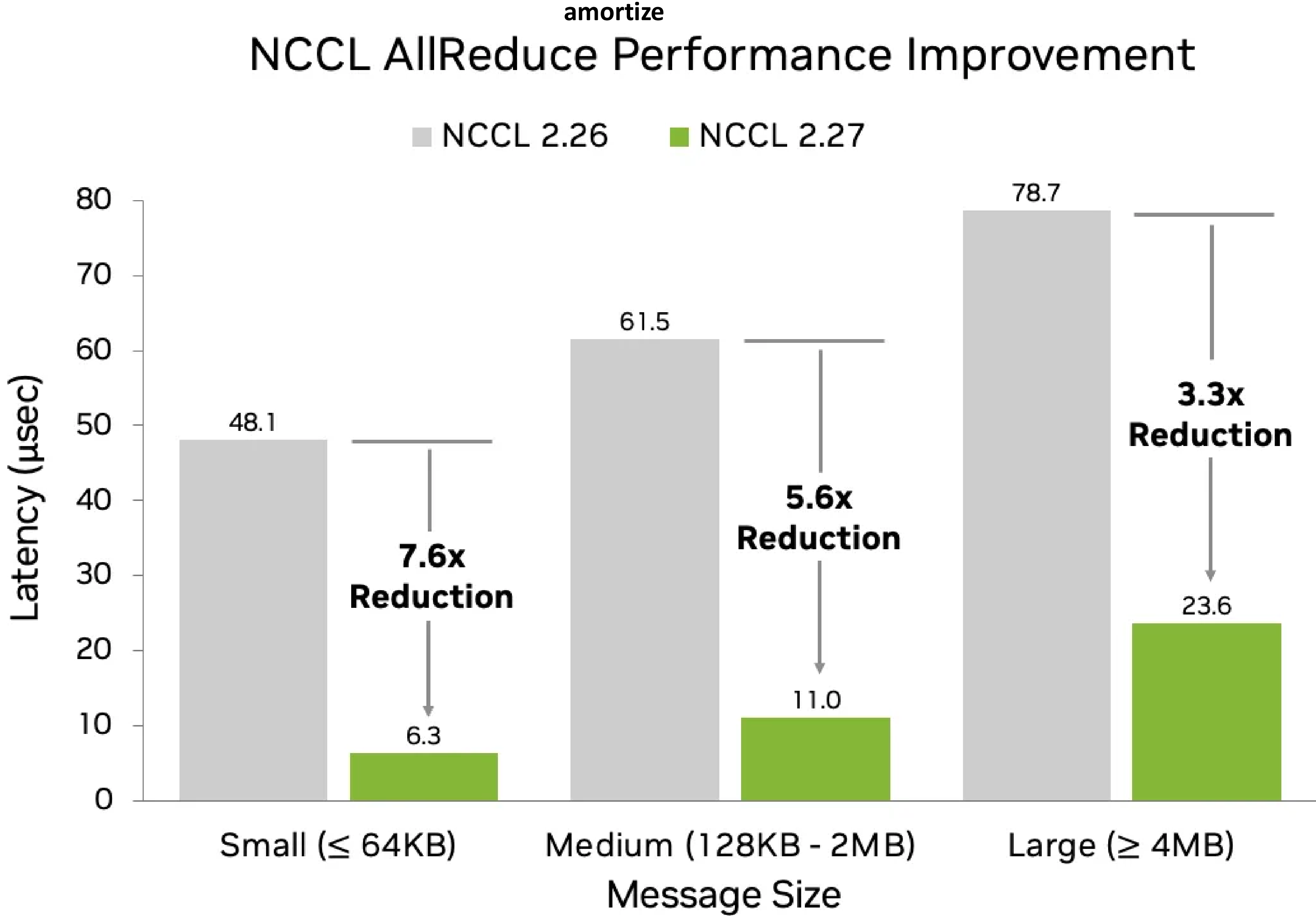
Tag matching
Active recv
Memory translation



With Symmetric:
Tag matching
Active recv
Memory translation

The resulting window is shared infrastructure for registered collectives, host PUT targets, and later device APIs.

Symmetric Memory Latency Impact



** Results from NVIDIA GB200, 32-Ranks. Lower is better.*

before the loop (once)
register compatible memory
on every rank

inside the loop (many times)
reuse the window for collective, PUT,
or device-API work

4a. Register symmetric memory into a window

The collective stays an AllGather; the operands become registered windows

- Exercise: 01-register-symmetric-window/

Register the source and destination, run the existing AllGather, then deregister only after the stream completes.

KEY APIs

```
ncclCommWindowRegister(comm, src, src_size,  
                        &src_win, NCCL_WIN_COLL_SYMMETRIC);  
ncclCommWindowRegister(comm, dst, dst_size,  
                        &dst_win, NCCL_WIN_COLL_SYMMETRIC);  
ncclAllGather(src, dst, src_size, ncclInt8, comm, stream);
```

After `cudaStreamSynchronize(stream)`, deregister both windows before freeing the allocations.

EFFECT

- Registration is collective: every rank supplies compatible source and destination regions.
- `ncclWindow_t` is a local opaque handle; the logical offsets match even when CUDA pointers differ.
- AllGather semantics do not change: dst receives rank-ordered source segments.

NCCL host PUT: one-sided, stream-ordered RMA

PUT writes a peer window; the signal hands off the payload

The target rank does not call ncclRecv. The source names the peer, registered window, and offset; the target queues a stream wait.

```
// source rank stream
ncclPutSignal(local, count, datatype,
              peer, dst_win, offset, 0, 0, 0, comm, stream);
// writes the peer window; its signal follows payload delivery

// target rank stream
ncclWaitSignal(1, &wait, comm, stream);
// after completion, the matching payload is visible to later work
```

- Unlike NVSHMEM, All NCCL Puts come with signals
- You must pass a registered window into a NCCL Put in addition to an offset

4b. Host PUT with symmetric operands

KEY APIs

```
ncclPutSignal(device_send, kElements, ncclInt,  
             destination, recv_window, 0,  
             kSignalIndex, kContext, 0, comm, stream);
```

```
ncclWaitSignalDesc_t wait = {1, source, kSignalIndex, kContext};  
ncclWaitSignal(1, &wait, comm, stream);
```

In this lab sigIdx = 0, ctx = 0, and flags = 0.

The target window is registered before the PUT.

EFFECT

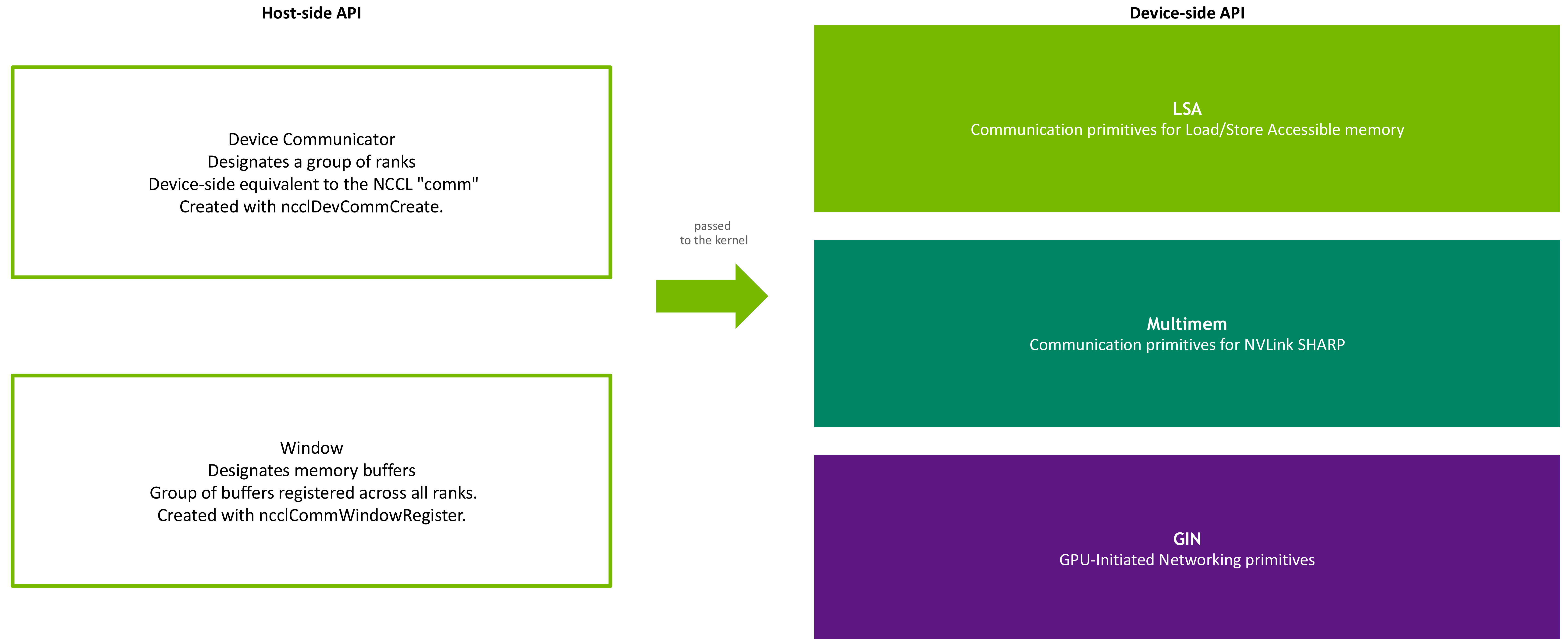
- destination does not post ncclRecv. PUT writes its registered recv_window and then updates its remote signal.
- When ncclWaitSignal completes, the matching payload is visible to later work on this stream.
- CPU return only means queued; the source is reusable after its stream reaches the PUT.



5. NCCL device APIs

NCCL Device API

In-kernel communication toolbox



NCCL Device API

In-kernel communication toolbox

LSA	Remote memory access →Get pointer for remote memory →Perform normal memory operations to pointer
Load/Store Accessible	Synchronization → LSA barrier
Multimem	NVLink SHARP access →Get pointer to multimem mappings
NVLink SHARP	→ Perform multimem operations on pointer
GIN	Remote memory access →Create GIN object, select GIN context →Perform put() operations →Wait on signals for remote operations
Gpu-Initiated Networking	→Wait on counters for local operations Synchronization →GIN barrier

```
__global__ kernel(ncclDevComm_t devComm, ncclWindow_t win) {

    float* ptr = ncclGetPeerPointer(devComm, offset, peer);
    ptr[0] = value;

    ncclBarrierSession bar {
        ncclCoopCta(), ncclTeamTagLsa(), devComm, blockIdx.x };
    bar.sync(ncclCoopCta(), cuda::memory_order_release);

    float* ptr = ncclGetLsaMultimemPointer(win, offset, devComm);
    value = multimem_reduce_sum(ptr);
    multimem_store(ptr, value);

    ncclGin gin { devComm, blockIdx.x % devComm.ginContextCount };
    gin.put(ncclTeamWorld(devComm), peer,
        win, remoteOffset, win, localOffset,
        size, ncclGin_SignalInc{signalIndex});
    gin.waitSignal(ncclCoopCta(), signalIndex, signalValue);

    ncclBarrierSession bar {
        ncclCoopCta(), ncclTeamTagWorld(), gin, blockIdx.x };
    bar.sync(ncclCoopCta(), cuda::memory_order_relaxed,
        ncclGinFenceLevel::relaxed);

}
```

NCCL Device API

In-kernel communication toolbox

LSA	Remote memory access →Get pointer for remote memory →Perform normal memory operations to pointer
Load/Store Accessible	Synchronization → LSA barrier

Multimem	NVLink SHARP access →Get pointer to multimem mappings → Perform multimem operations on pointer
NVLink SHARP	

GIN	Remote memory access →Create GIN object, select GIN context →Perform put() operations →Wait on signals for remote operations
Gpu-Initiated Networking	→Wait on counters for local operations Synchronization →GIN barrier

```
__global__ kernel(ncclDevComm_t devComm, ncclWindow_t win) {

    float* ptr = ncclGetPeerPointer(devComm, offset, peer);
    ptr[0] = value;

    ncclBarrierSession bar {
        ncclCoopCta(), ncclTeamTagLsa(), devComm, blockIdx.x };
    bar.sync(ncclCoopCta(), cuda::memory_order_release);

    float* ptr = ncclGetLsaMultimemPointer(win, offset, devComm);
    value = multimem_reduce_sum(ptr);
    multimem_store(ptr, value);

    ncclGin gin { devComm, blockIdx.x % devComm.ginContextCount };
    gin.put(ncclTeamWorld(devComm), peer,
        win, remoteOffset, win, localOffset,
        size, ncclGin_SignalInc{signalIndex});
    gin.waitSignal(ncclCoopCta(), signalIndex, signalValue);

    ncclBarrierSession bar {
        ncclCoopCta(), ncclTeamTagWorld(), gin, blockIdx.x };
    bar.sync(ncclCoopCta(), cuda::memory_order_relaxed,
        ncclGinFenceLevel::relaxed);

}
```

NCCL Device API

In-kernel communication toolbox

LSA	Remote memory access →Get pointer for remote memory →Perform normal memory operations to pointer
Load/Store Accessible	Synchronization → LSA barrier

Multimem	NVLink SHARP access →Get pointer to multimem mappings → Perform multimem operations on pointer
NVLink SHARP	

GIN	Remote memory access →Create GIN object, select GIN context →Perform put() operations →Wait on signals for remote operations →Wait on counters for local operations
Gpu-Initiated Networking	Synchronization →GIN barrier

```
__global__ kernel(ncclDevComm_t devComm, ncclWindow_t win) {

    float* ptr = ncclGetPeerPointer(devComm, offset, peer);
    ptr[0] = value;

    ncclBarrierSession bar {
        ncclCoopCta(), ncclTeamTagLsa(), devComm, blockIdx.x };
    bar.sync(ncclCoopCta(), cuda::memory_order_release);

    float* ptr = ncclGetLsaMultimemPointer(win, offset, devComm);
    value = multimem_reduce_sum(ptr);
    multimem_store(ptr, value);

    ncclGin gin { devComm, blockIdx.x % devComm.ginContextCount };
    gin.put(ncclTeamWorld(devComm), peer,
        win, remoteOffset, win, localOffset,
        size, ncclGin_SignalInc{signalIndex});
    gin.waitSignal(ncclCoopCta(), signalIndex, signalValue);

    ncclBarrierSession bar {
        ncclCoopCta(), ncclTeamTagWorld(), gin, blockIdx.x };
    bar.sync(ncclCoopCta(), cuda::memory_order_relaxed,
        ncclGinFenceLevel::relaxed);

}
```

NCCL Device API

In-kernel communication toolbox

LSA	Remote memory access →Get pointer for remote memory →Perform normal memory operations to pointer
Load/Store Accessible	Synchronization → LSA barrier

Multimem	NVLink SHARP access →Get pointer to multimem mappings → Perform multimem operations on pointer
NVLink SHARP	

GIN	Remote memory access →Create GIN object, select GIN context →Perform put() operations →Wait on signals for remote operations →Wait on counters for local operations
Gpu-Initiated Networking	Synchronization →GIN barrier

```
__global__ kernel(ncclDevComm_t devComm, ncclWindow_t win) {

    float* ptr = ncclGetPeerPointer(devComm, offset, peer);
    ptr[0] = value;

    ncclBarrierSession bar {
        ncclCoopCta(), ncclTeamTagLsa(), devComm, blockIdx.x };
    bar.sync(ncclCoopCta(), cuda::memory_order_release);

    float* ptr = ncclGetLsaMultimemPointer(win, offset, devComm);
    value = multimem_reduce_sum(ptr);
    multimem_store(ptr, value);

    ncclGin gin { devComm, blockIdx.x % devComm.ginContextCount };
    gin.put(ncclTeamWorld(devComm), peer,
        win, remoteOffset, win, localOffset,
        size, ncclGin_SignalInc{signalIndex});
    gin.waitSignal(ncclCoopCta(), signalIndex, signalValue);

    ncclBarrierSession bar {
        ncclCoopCta(), ncclTeamTagWorld(), gin, blockIdx.x };
    bar.sync(ncclCoopCta(), cuda::memory_order_relaxed,
        ncclGinFenceLevel::relaxed);

}
```

Four objects tell the kernel who, where, and how

Host setup creates them; device code composes them



Team chooses ranks.
Coop chooses threads.

```
__global__ void kernel(ncclDevComm dev_comm, ncclWindow_t payload)
```

The kernel receives these handles. It does not create a communicator.

ncclDevComm rank, size, topology, and reserved device resources

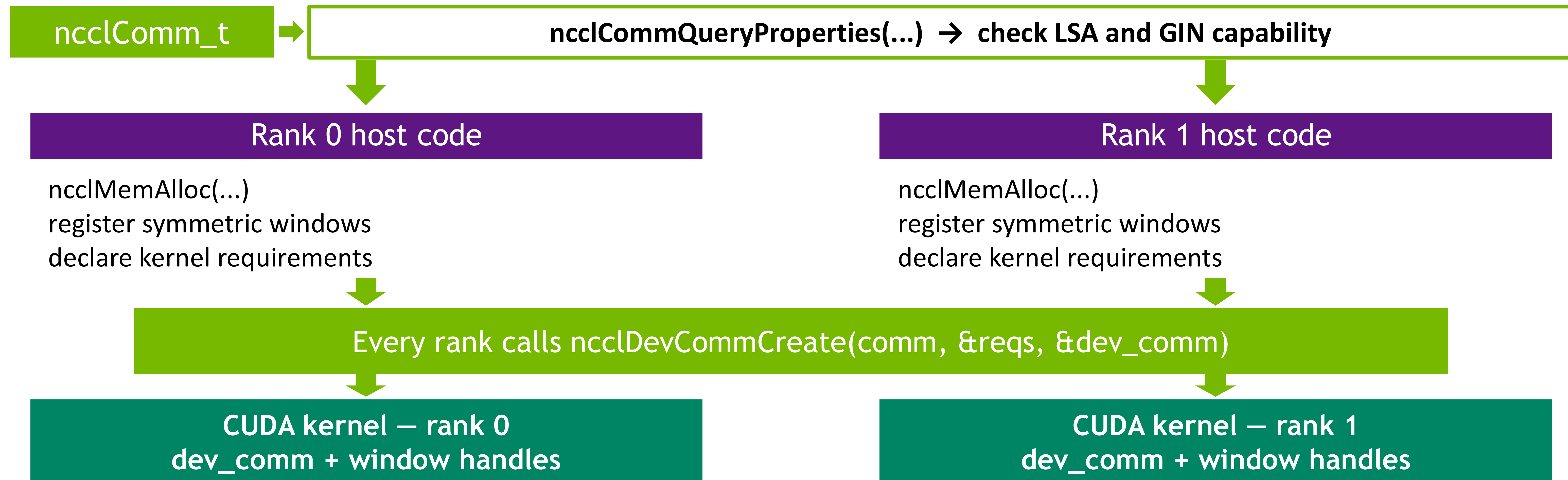
ncclWindow_t registered allocation addressed by byte offset

ncclTeamWorld / ncclTeamLsa which ranks participate

ncclCoopThread / Warp / Cta which GPU threads call together

The host prepares the objects the kernel will use

Setup is collective; kernels receive a device communicator and window handles

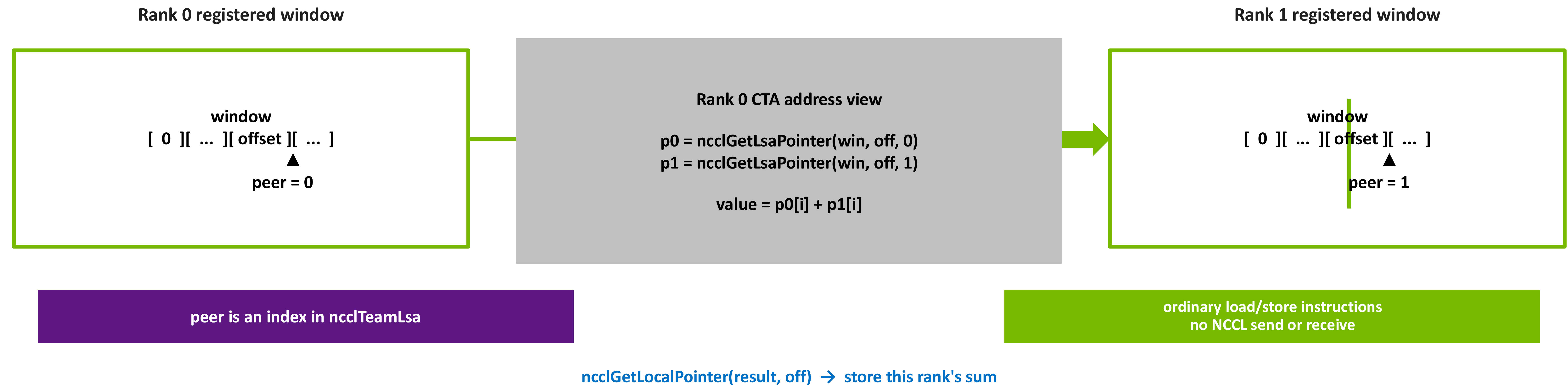


After stream completion: destroy dev comm → deregister windows → free memory

```
ncclDevCommRequirements reqs = NCCL_DEV_COMM_REQUIREMENTS_INITIALIZER;  
ncclDevCommCreate(comm, &reqs, &dev_comm);  
kernel<<<grid, block, 0, stream>>>(dev_comm, payload_window, ...);
```

LSA gives each CTA pointers into every peer's window

The window, byte offset, and LSA-team rank resolve to a locally accessible address



```
bar.sync(ncclCoopCta(), cuda::memory_order_acquire);  
auto *peer = ncclGetLsaPointer(source_window, byte_offset, lsa_peer);  
auto *local = ncclGetLocalPointer(result_window, byte_offset);  
bar.sync(ncclCoopCta(), cuda::memory_order_release);
```

Exercise 5a: LSA device API

Build a two-rank all-reduce with direct LSA loads

Inside one CTA, synchronize the LSA team, load both ranks' source windows, store the local sum, and synchronize again.

[05-nccl-device-apis/01-lsa-device-api/](#)

ncclLsaBarrierSession<ncclCoopCta> bar{...}

Synchronizes participating CTAs across the LSA team.

ncclGetLsaPointer(source_window, 0, lsa_peer)

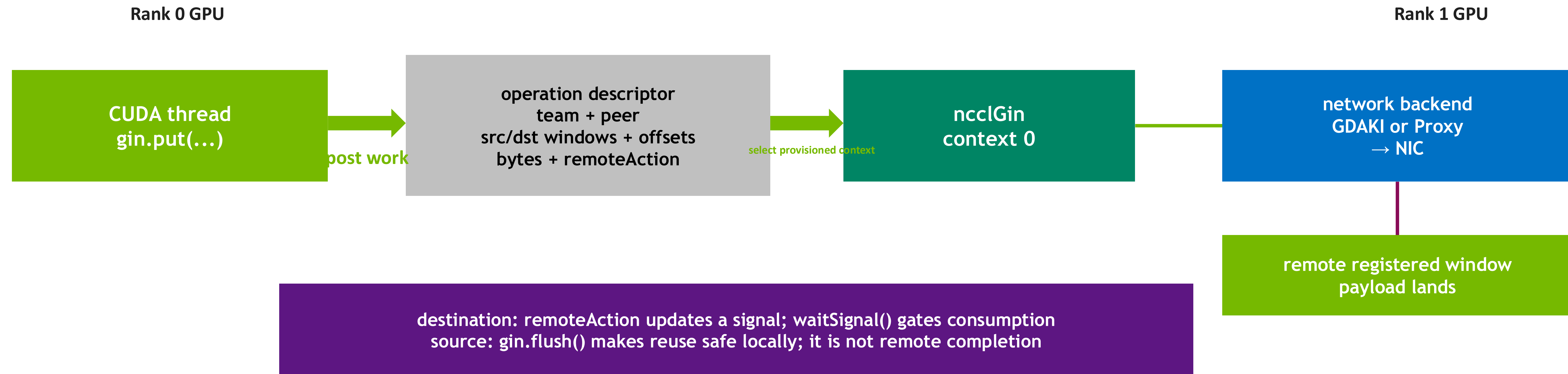
Returns a pointer the kernel can load directly.

ncclGetLocalPointer(result_window, 0)

Returns this rank's registered result pointer; the release barrier closes the session.

GIN posts network work from a CUDA thread

A put descriptor flows through a provisioned GIN context to the NIC and remote window



```
ncclGin gin{dev_comm, 0}; // selects a context provisioned by ncclDevCommCreate
gin.put(team, peer, dst_win, dst_off, src_win, src_off, bytes, remote_action);
gin.waitSignal(ncclCoopCta(), signal_window, 0, expected);
```

Exercise 5b: GIN put device API

Rank 0 puts 28; rank 1 waits for the remote signal

Complete the local source store, gin.put, remote signal action, and target-side wait.

05-nccl-device-apis/02-gin-put-device-api/

```
ncclGin gin(dev_comm, 0);
```

Creates device-side networking context 0.

```
gin.put(world, 1, payload_window, 0, payload_window, 0,  
        sizeof(kPutValue), ncclGin_VASignalInc{signal_window, 0});
```

The same window names rank 0's source and rank 1's matching destination.

```
gin.waitSignal(ncclCoopCta(), signal_window, 0, 1);
```

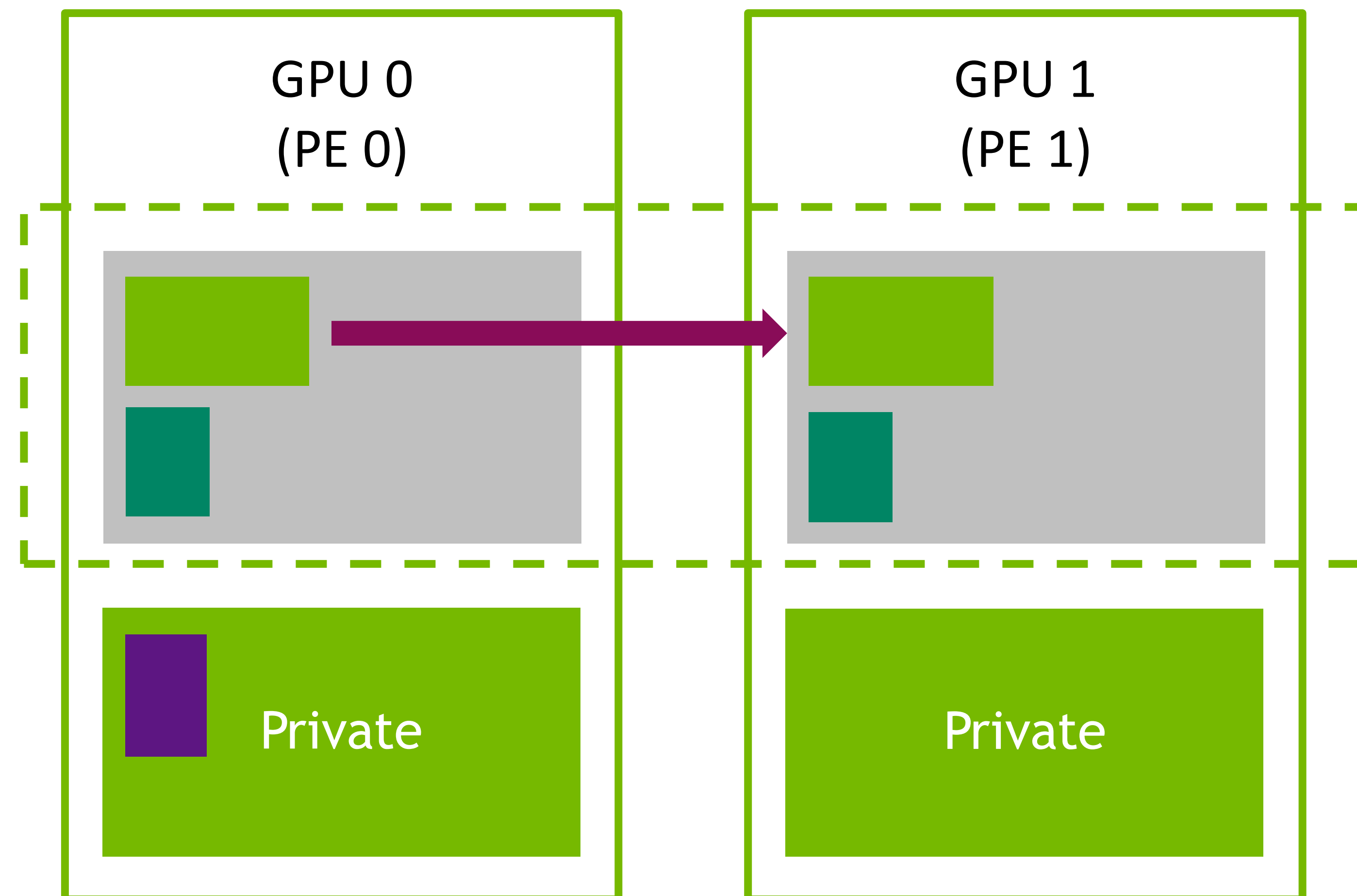
Rank 1 waits until this put's remote signal is visible.



6. Python APIs

NVSHMEM4Py Host API

Host/Device Put



Same as the Put you are used to, but from Python!

```
# Host APIs take a CUDA Stream as an argument to allow pre-orchestrating dependencies
nvshmem.core.put(dst: Array, src: Array, pe: Integer, stream: cuda.core.Stream)
```

6a. NVSHMEM4Py Host APIs

NVSHMEM4Py

```
NVSHMEM: 01-nvshmem4py-host-apis/
```

```
nvshmem.core.put(...,  
    stream=stream)  
put_signal(...); signal_wait(...)
```

NCCL Host APIs in Python

(This slide should look pretty familiar)

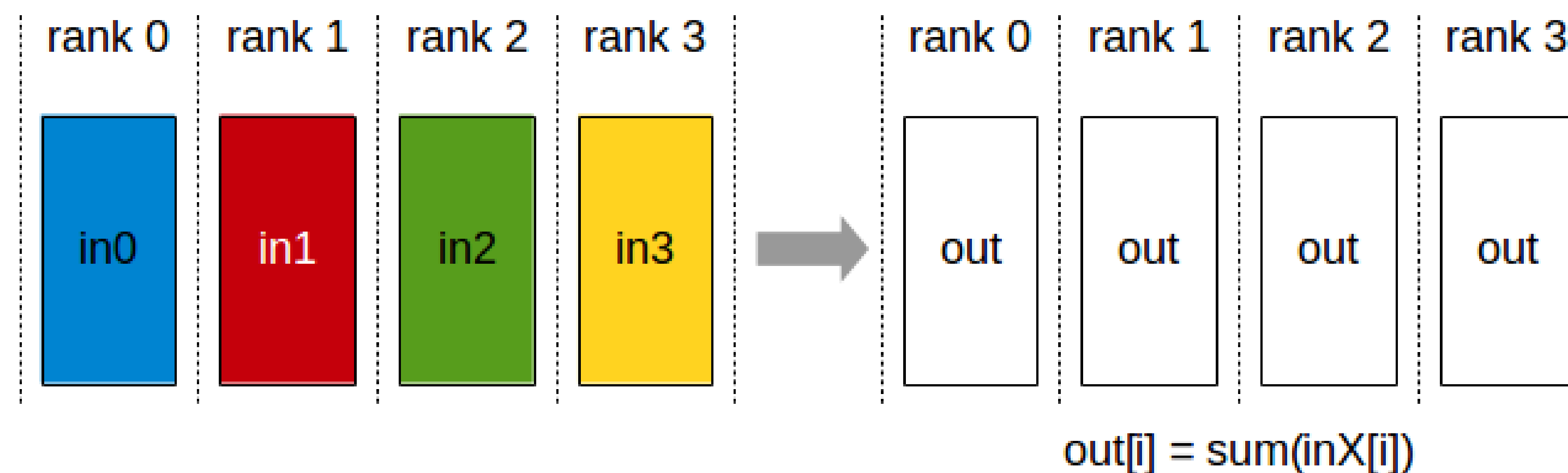
- Send/Recv

```
nccl.Communicator.send(sendbuf: NcclBufferSpec, peer: int, *, stream: NcclStreamSpec | None = None);  
nccl.Communicator.recv(recvbuf: NcclBufferSpec, peer: int, *, stream: NcclStreamSpec | None = None);
```

- Collective Operations

```
nccl.Communicator.allreduce(sendbuf: NcclBufferSpec, recvbuf: NcclBufferSpec, op: NcclRedOp | CustomRedOp,  
                             stream: NcclStreamSpec | None = None);  
nccl.Communicator.broadcast(sendbuf: NcclBufferSpec | Any, recvbuf: NcclBufferSpec, root: int, *,  
                             stream: NcclStreamSpec | None = None);  
nccl.Communicator.reduce(sendbuf: NcclBufferSpec, recvbuf: NcclBufferSpec | Any, op: NcclRedOp | CustomRedOp,  
                          root: int, *, stream: NcclStreamSpec | None = None);
```

- Allreduce example



6b. NCCL4Py Host APIs

NCCL4Py

NCCL4Py: `allreduce.py`, `send_recv.py`
02-nccl4py-host-apis/

```
nccl_comm.reduce(data, data, nccl.SUM)
```

```
with nccl.group():
```

```
    nccl_comm.send(...); nccl_comm.recv(...)
```

Custom Communication Kernels

When to write one



Prefer using on-stream APIs!

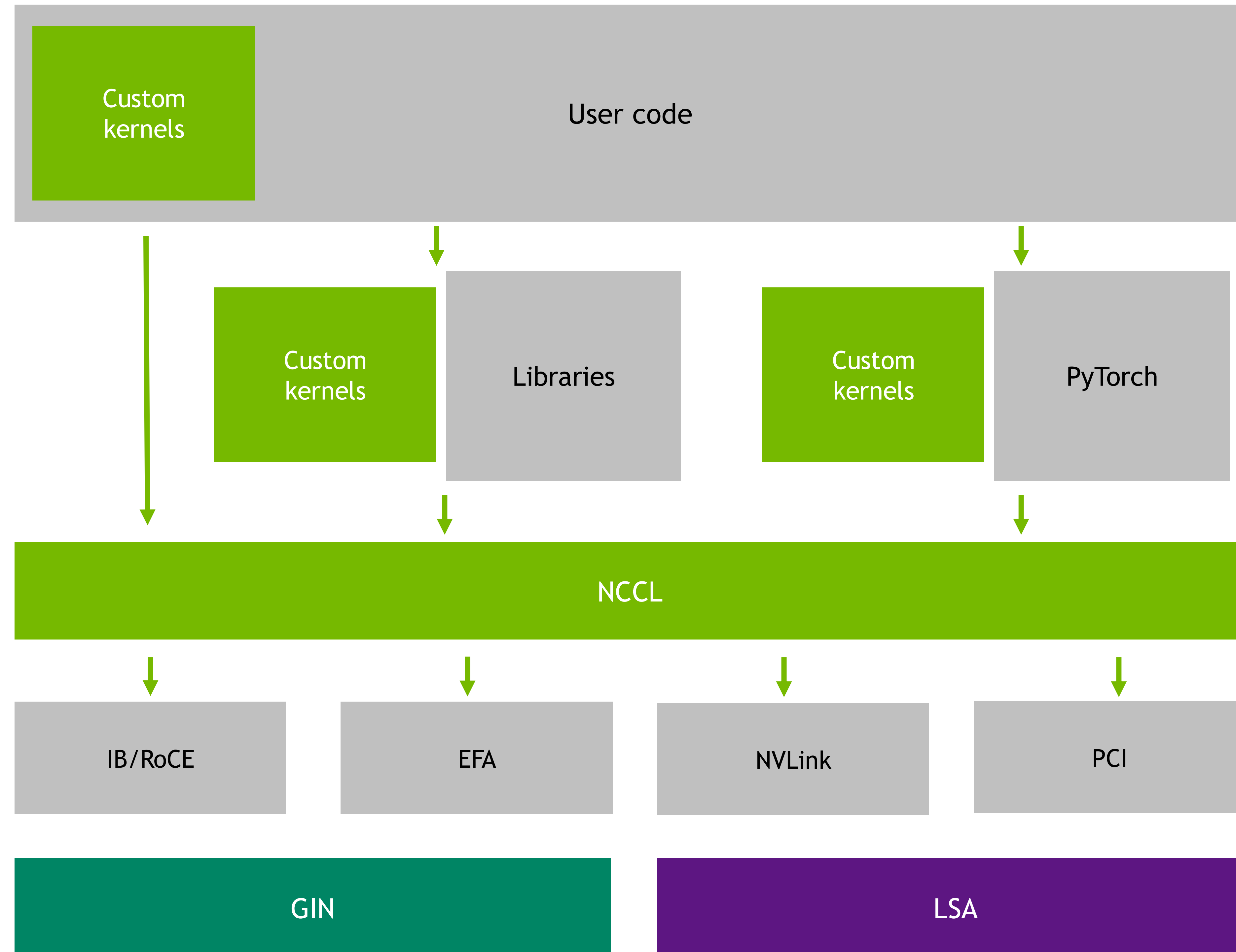
But... Write custom kernels when:

- Compute pattern is unique
- Finer control on threads/warp/block
- Fusion is desired

Historically: C/C++ only

Custom Communication Kernels

Integration stack



Numba CUDA

Vectorized Adds

- “CUDA kernel in Python”
 - JIT (Just-in-time) compiled from (a subset of) Python
 - Interoperable with common array/tensor libraries
-
- Usage:
 - [Go 200,000x Faster in the Field of Weather Analysis with CUDA Python \(Numba\) | by Chen Chieh Yu | RAPIDS AI | Medium](#)
 - [Multi-blank transducer loss \(Forward / Backward\) | Nvidia Nemo](#)

```
import numpy as np
from numba import cuda
```

```
N = 1_000_000
```

```
@cuda.jit
def vector_add(r, x, y):
    i = cuda.grid(1)
    if i < len(r):
        r[i] = x[i] + y[i]
```

```
x = np.random.rand(N).astype(np.float32)
y = np.random.rand(N).astype(np.float32)
r = cuda.device_array(N, dtype=np.float32)
```

```
vector_add[((N + 255) // 256), 256](r, x,
y)
```

CUTLASS

Programming Tensor Cores

Where it's used

- External: PyTorch/vLLM/SGLang, FlashAttention, Quack, MoonEP
- Internal: cuDNN, FlashInfer, Transformer Engine, TRT LLM

What's New: v4.7.0

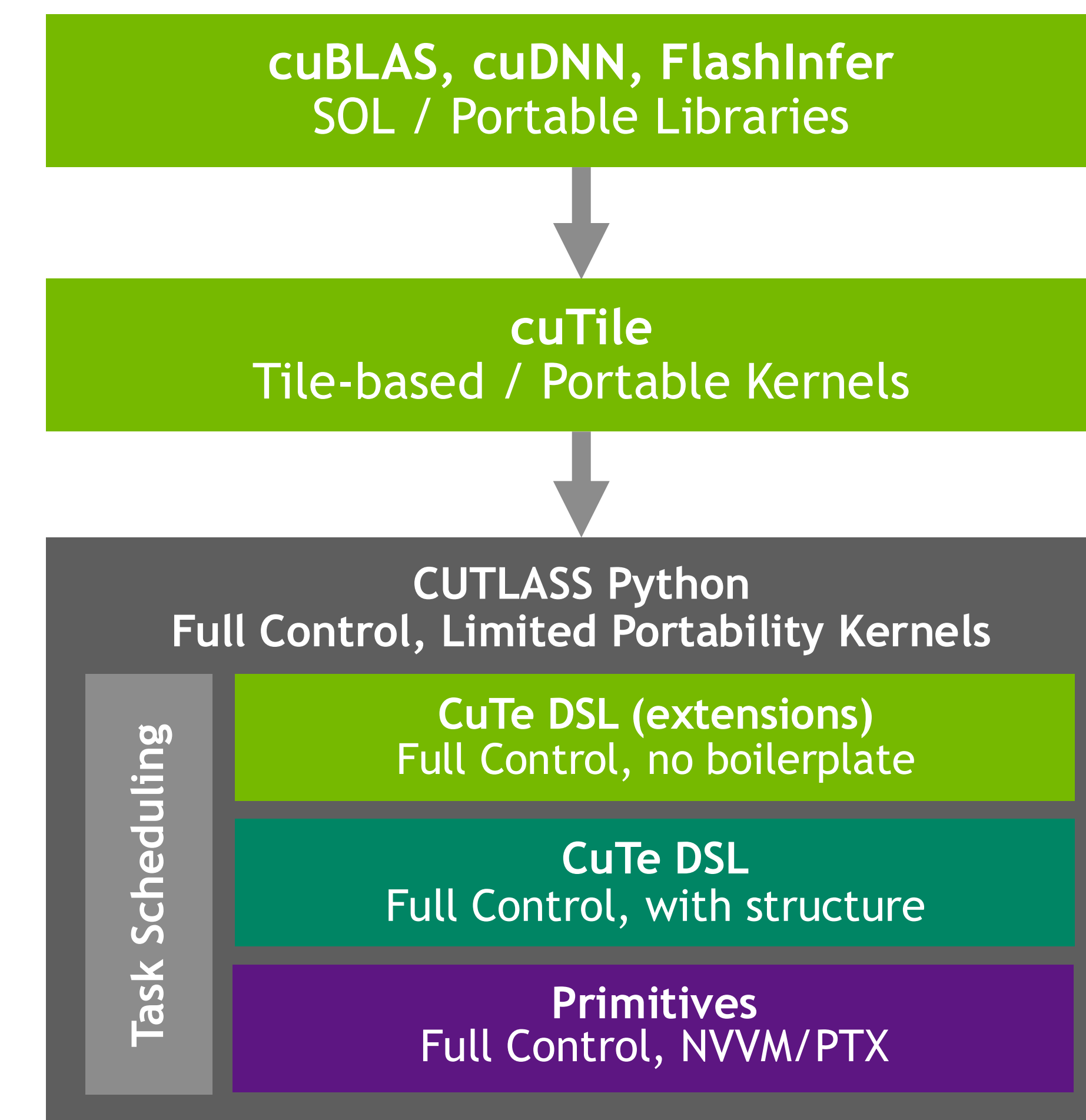
- **Primitives:** SIMT programming to simplify non-GEMM-like kernels
- **Task Scheduling:** Eliminate classes of concurrency bugs at compile-time
- **CUTLASS Profiler in Python:** Surface the best CUTLASS Python kernel for customization

Capabilities

- **Matrix-Matrix Multiplication (GEMM)** - The foundation of AI and deep learning computations
- **Convolution Operations** - Through implicit GEMM algorithms
- **Mixed-Precision Computing** - Supporting FP64, FP32, TF32, FP16, BF16, FP8, FP6, FP4 and integer data types
- **Tensor Core Programming** - Optimized for NVIDIA Ampere through Rubin architectures
- **Language:** C++ header library, Python DSLs

Value / Take-Away

- Co-designed access to full Tensor Core performance
- Multiple abstraction-levels to balance developer velocity and performance



NVIDIA's kernel authoring software stack with CUTLASS 4.7

Compiling Python to GPU Code

- Three components are required for writing device code:
 - Device kernel definition (Device code)
 - Compilation control (Host code)
 - Launch control (Host code)

<pre>1 import cutlass 2 import cutlass.cute as cute 3 4 @cute.kernel 5 def hello_world(): 6 tid, _, _ = cute.arch.thread_idx() 7 if tid == 0: 8 cute.printf("Hello, world from GPU!") 9</pre>	Device code (Compiled)
<pre>10 @cute.jit 11 def hello_world_launcher(): 12 nblocks = 32 13 threads_per_block = 512 14 hello_world().launch(15 grid=[nblocks, 1, 1], 16 block=[threads_per_block, 1, 1], 17) 18</pre>	Host code (Launch)
<pre>19 hello_world_compiled = hello_world_launcher.compile() 20 hello_world_compiled()</pre>	Host code (Compile)

Custom Kernel Execution with CuTe DSL

```
1 import nvshmem.core
2 from cuda.core import Device, system
3 import nvshmem.core.device.cute as nvshmem_cute
4
5 @cute.kernel
6 def put_signal_kernel(dst: cute.Tensor, src: cute.Tensor, signal_var: cute.Tensor):
7     tid, _, _ = cute.arch.thread_idx()
8     send_peer = (nvshmem_cute.my_pe() + 1) % nvshmem_cute.n_pes()
9     rcv_peer = (nvshmem_cute.my_pe() - 1) % nvshmem_cute.n_pes()
10
11     # CuTe DSL Compute code can go here
12
13     if tid == 0:
14         nvshmem_cute.put_signal(dst, src, signal_var, 1, nvshmem.core.SignalOp.SIGNAL_SET, peer)
15         nvshmem_cute.signal_wait(signal_var, nvshmem.core.ComparisonType.CMP_GE, signal_val)
16
17 @cute.jit
18 def put_signal_launcher(dst: nvshmem.core.Tensor, src: nvshmem.core.Tensor, signal_var: nvshmem.core.Tensor):
19     test_put_signal_with_wait(dst, src, signal_var).launch(
20         grid=[32, 1, 1],
21         block=[cute.size(WARP_SIZE, mode=[0]), 1, 1],
22     )
23
24 # Allocate resources
25 dev = Device()
26 stream = dev.create_stream()
27 buf_src = nvshmem.core.tensor((4, 4), dtype=torch.float32)
28 buf_src[:] = nvshmem.core.my_pe() + 1
29 buf_dst = nvshmem.core.tensor((4, 4), dtype=torch.float32)
30 buf_dst[:] = 0
31 signal_var = nvshmem.core.tensor((1, ), dtype=torch.int64)
32
33 # Compile and run kernel
34 compiled = _compile_kernel(put_signal_launcher, buf_dst, buf_src, signal_var)
35 compiled(buf_dst, buf_src, signal_var)
```

- Python → CUTLASS compiler → CUDA executable
- Kernel launch parameters specified by @cute.jit below
- Exposes hardware knobs (e.g. Tensor Core)

- Dispatched to host compiler
- Evaluates to CUDA driver + library calls
- Kicks off search for @cute.kernel



- Torch parts get executed in C++ via pybind11

DSL Compilation and Linking

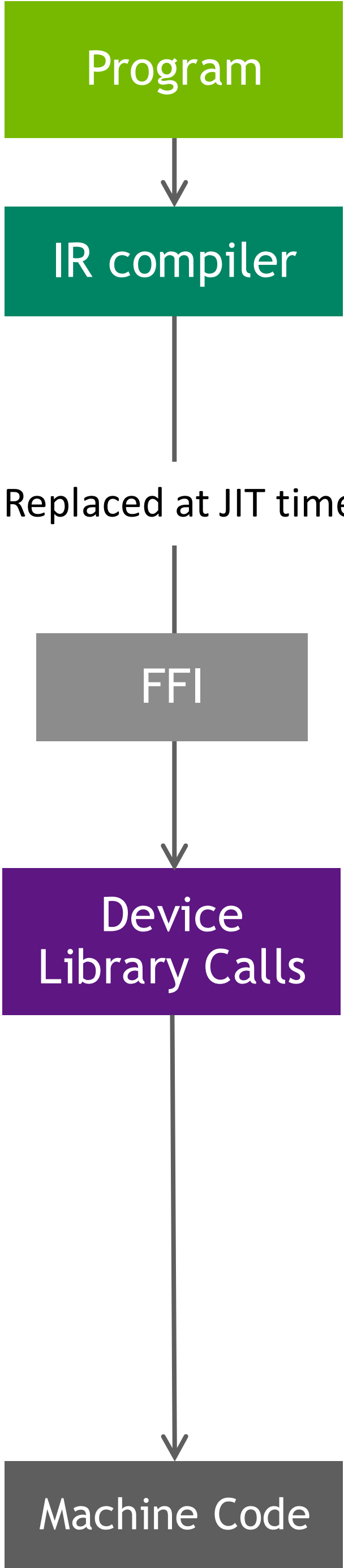
Numba CUDA

```
@cuda.jit(lto=True)
def ring_shift_kernel(dest):
    mype = my_pe()
    npes = n_pes()
    peer = int32((mype + 1) % npes)

    nvshmem_numba.p(dest, mype, peer)
```

CuTe DSL

```
1 import nvshmem.core.device.cute as nvshmem_cute
2
3 @cute.kernel
4 def simple_shift_kernel(destTensor: cute.Tensor):
5     tid, _, _ = cute.arch.thread_idx()
6     mype = nvshmem_cute.my_pe()
7     npes = nvshmem_cute.n_pes()
8     peer = (mype + 1) % npes
9     if tid == 0:
10         nvshmem_cute.p(destTensor, mype, peer)
```



```
_Z14nvshmem_long_p_nbst = declare_device(
    '_Z14nvshmem_long_p_nbst',
    void(
        CPointer(CPointer(int64)), CPointer(int64), CPointer(int32)
    )
)

shim_raw_str = ""
extern "C" __device__ int
_Z14nvshmem_long_p_nbst(int &retval, long ** dest, long* value, int* pe) {
    nvshmem_long_p(*dest, *value, *pe);
    return 0;
}

.....
```

```
int_p = cute.ffi(name="nvshmem_int_p", params_types=[...], return_type=None)
```

JIT Linked with JIT compiled device lib
Automatically linked/loaded
Implicitly compiled

```
dest = nvshmem.core.array((1, ), dtype="int32")

ring_shift_kernel[1, 1, 0](dest)

barrier_all()
dev.sync()

print(f"{h_my_pe():} received message {dest[0]}")

nvshmem.core.free_array(dest)
nvshmem.core.finalize()
```

JIT Linked with pre-compiled device lib
Launched against host lib
Explicitly compiled

```
nvshmem_device_bc = f"{os.environ['NVSHMEM_HOME']}/src/lib/libnvshmem_device.bc"

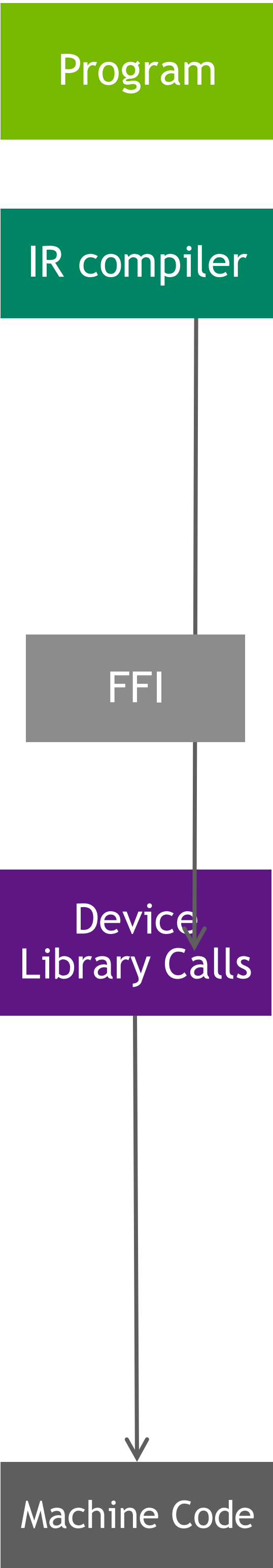
compilerd_func = cute.compile(
    simple_shift,
    tensor_dlpack,
    options=f" --link-libraries={nvshmem_device_bc}",
)

compilerd_func = compilerd_func.to(my_pe)
cuda_library = compilerd_func.jit_module.cuda_library
nvshmem_kernel = nvshmem.core.NvshmemKernelObject.from_handle(int(cuda_library[0]))
nvshmem.core.library_init(nvshmem_kernel)
```

NCCL DSL APIs

- NCCL device API is available in CUTLASS-Python
- Host-side APIs come from nccl4py

```
1
2 nccl_comm = nccl.Communicator.init(nranks=nranks, rank=rank, unique_id=unique_id)
3
4 send_buf = nccl.cupy.empty(NUM_ELEMS, dtype='int64')
5 recv_buf = nccl.cupy.empty(NUM_ELEMS, dtype='int64')
6
7 send_win = nccl_comm.register_window(send_buf)
8 recv_win = nccl_comm.register_window(recv_buf)
9
10 reqs = nccl.NCCLDevCommRequirements(
11     gin_connection_type=nccl.NcclGinConnectionType.FULL,
12     gin_signal_count=SIGNSAL_ID + 1,
13 )
14 dev_comm = nccl_comm.create_dev_comm(requirements=reqs)
15
16 test_nccl_put(dev_comm.ptr, send_win.handle, recv_win.handle)
```



CuTe DSL

```
1 import nccl.core.device.cute as nccl_cute
2
3 @cute.kernel
4 def test_nccl_put_kernel(dev_comm, send_win, recv_win):
5     tid, _, _ = cute.arch.thread_idx()
6     dev_comm = nccl_cute.DevComm(dev_comm)
7     send_win = nccl_cute.Window(send_win)
8     recv_win = nccl_cute.Window(recv_win)
9     team = dev_comm.team_world
10    gin = dev_comm.gin(nccl_cute.GinBackendMask.ALL, 0)
11    coop = nccl_cute.cta()
12
13    # cute.Tensor views spanning the full 1 MiB of each window.
14    send = send_win.tensor(cutlass.Int64, cute.make_layout(NUM_ELEMS))
15    recv = recv_win.tensor(cutlass.Int64, cute.make_layout(NUM_ELEMS))
16
17    if team.nRanks > 2:
18        if 0 == team.rank:
19            if 0 == tid:
20                cute.printf(f"Before Put: send[0]={send[0]} send[{NUM_ELEMS - 1}]={send[ NUM_ELEMS - 1 ]}\n")
21                gin.put( team, DST_RANK, recv_win, recv, # destination window + tensor (lives on the peer)
22                        send_win, send, # source window + tensor (local)
23                        coop, is_signal=True, signal_id=SIGNSAL_ID, signal_op=0, signal_op_arg=1,
24                        )
25            if 1 == team.rank:
26                gin.wait_signal(coop, signal=SIGNSAL_ID, least=1)
27            if 0 == tid:
28                cute.printf(f"After Put:  recv[0]={recv[0]} recv[{NUM_ELEMS - 1}]={recv[ NUM_ELEMS - 1 ]}\n")
```

JIT Linked with pre-compiled device lib
Launched against host lib

```
1 test_nccl_put_kernel(dev_comm, send_win, recv_win).launch(
2     grid=[1, 1, 1],
3     block=[cute.size(WARP_SIZE, mode=[0]), 1, 1],
4     cooperative=True
5 )
```

CuTe DSL Support for NCCL

Compute / comm in one kernel

- Effectively “Zero additional SM”

Auto JIT optimized

```
import cutlass.cute as cute
from nccl.core.device.cute import core, gin, coop

@cute.kernel
def gin_put_kernel(dev_comm, window):
    team = core.team_world(dev_comm)
    gin_ctx = gin.init(dev_comm, gin.GinBackendMask.ALL, 0)
    coop_ctx = coop.init_cta()
    if team.rank == 0:
        gin.put(gin_ctx, coop_ctx, team, dst_rank, window, offset,
                signal_id=1, signal_op=0, ...)
    elif team.rank == 1:
        gin.wait_signal(gin_ctx, coop_ctx, signal=1, least=1)
```

Exercise 6c: NCCL ring put from CuTe

Complete the **two TODOs in ring_put_kernel** using the APIs below.

```
gin.put(  
    team, peer, dst_win, dst, src_win, src, coop, *,  
    is_signal=False, signal_id=0,  
    signal_op=0, signal_op_arg=0,  
)  
"""Copy src from the local registered window to dst in peer's  
registered window; optionally update signal_id after the payload completes."""  
  
gin.wait_signal(  
    coop, *, signal=0, least=1,  
)  
"""Wait until the local GIN signal slot reaches the requested minimum."""
```

Exercise 6c: NVSHMEM ring put from CuTe

Complete the **TODO** in `ring_put_kernel` using the APIs below.

```
cute_my_pe() -> int
"""Return the calling PE's rank in the world team."""

cute_n_pes() -> int
"""Return the number of PEs in the world team."""

cute_int_p(dest, value, pe) -> None
"""Write one integer value to symmetric address dest on PE pe."""
```



NCCL Contrib and Extensions

NCCL is an ecosystem, not just a library

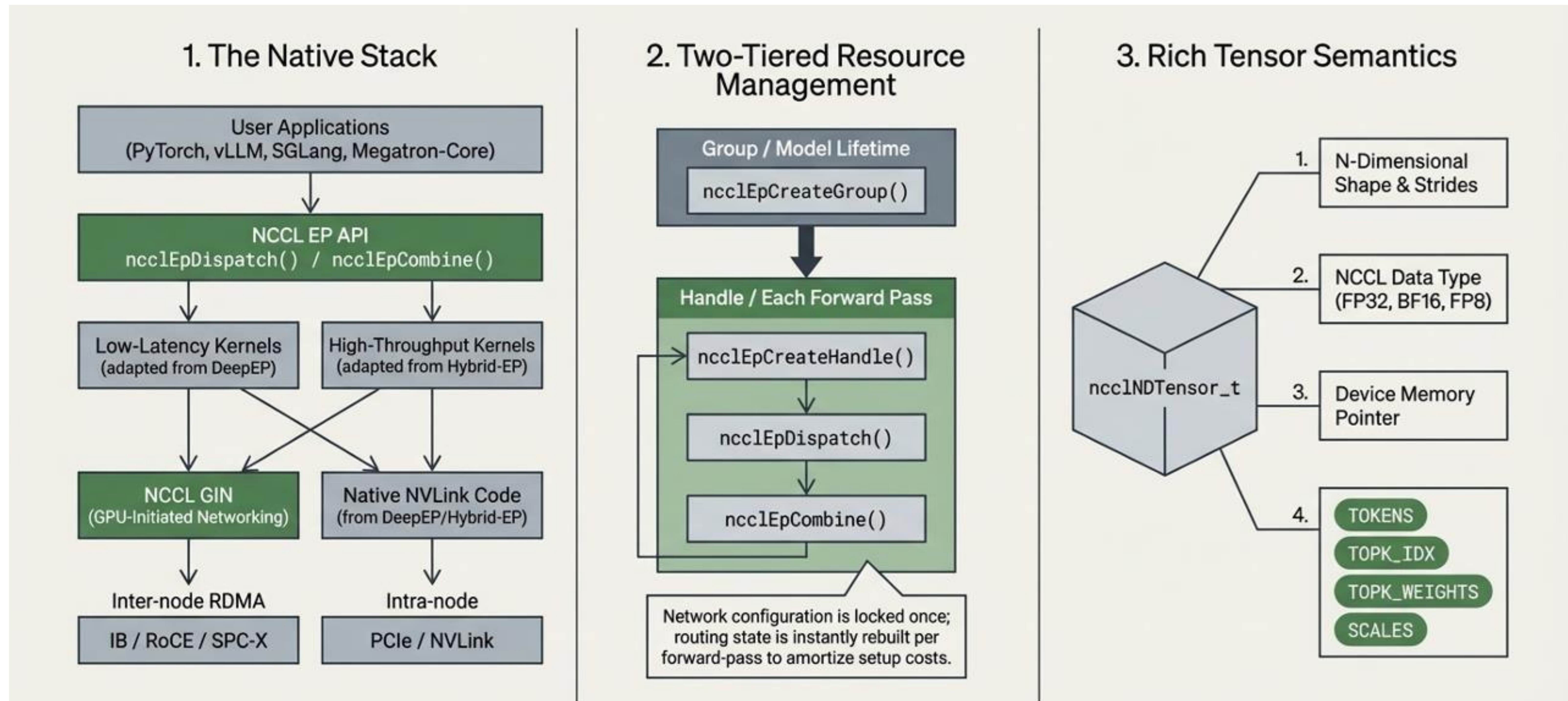
Public host and device APIs are a foundation for experiments, language bindings, and workload libraries

NCCL core provides the transports and communication primitives. Two adjacent project areas build on that API surface without turning every new idea into a core NCCL feature.



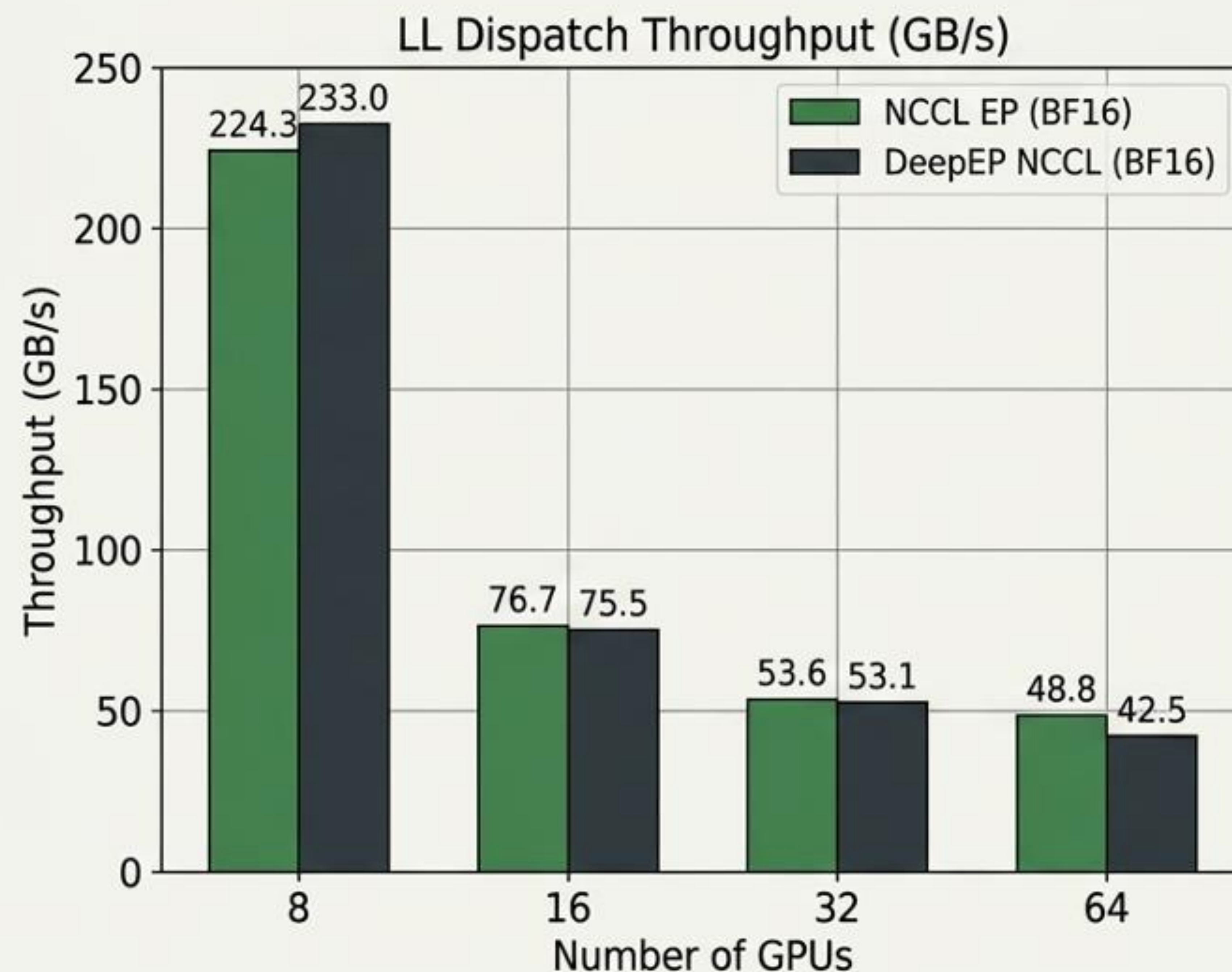
NCCL EP: Unified Architecture & Feature-Complete API

Native stack, two-tiered resource management, and rich N-dimensional tensor semantics



Performance Parity: Matching State-of-the-Art Dispatch Throughput

NCCL EP matches or exceeds DeepEP at 2–8 nodes, reaching 48.8 GB/s at 64 GPUs



NCCL EP matches or exceeds DeepEP at multi-node scales (2–8 nodes), achieving a notable throughput advantage at 64 GPUs (48.8 GB/s vs 42.5 GB/s).

Testing Context

Hardware: H100 EOS Cluster
Experts: 256
Tokens: 128
Routing: top-k 8

Exercise 7a: use NCCL EP

Mixture-of-experts routing through the Python Group / Handle API

tokens → dispatch → local expert work → combine → source-token order

Starter: [07-nccl-contrib/01-nccl-ep/ep_test.py](#)

Only the four calls below are TODOs. Setup, routing data, buffers, expert work, checks, and teardown are supplied.

CREATE GROUP

```
nccl_ep.Group.create(
    comm, config
) -> Group

"""Collectively create an EP group over an existing NCCL
communicator."""
```

CREATE HANDLE

```
group.create_handle(
    layout, topk_idx, *,
    layout_info=None,
    config=None, stream
) -> Handle

"""Bind the routing layout and state used by dispatch and
combine."""
```

MOVE TOKENS

```
handle.dispatch(
    inputs, outputs, *,
    layout_info=None, config=None, stream
)
"""Route tokens to selected expert-owner ranks."""

handle.combine(
    inputs, outputs, *, config=None, stream
)
"""Return results to source tokens in original order."""
```

NCCL and NVSHMEM projects

Contrib and extensions

NCCL CONTRIB

NVIDIA/nccl · staging/contrib

custom_algos
nccl4rust
nccl_checkpoint
nccl_ubx
niin

NCCL EXTENSIONS

NVIDIA/nccl-extensions

nccl_ep
nccl_m2n
python (bindings)

NVSHMEM CONTRIB

NVIDIA/nvshmem · devel/contrib

nvshmem4rust



Applications



Jacobi solver

Jacobi solver

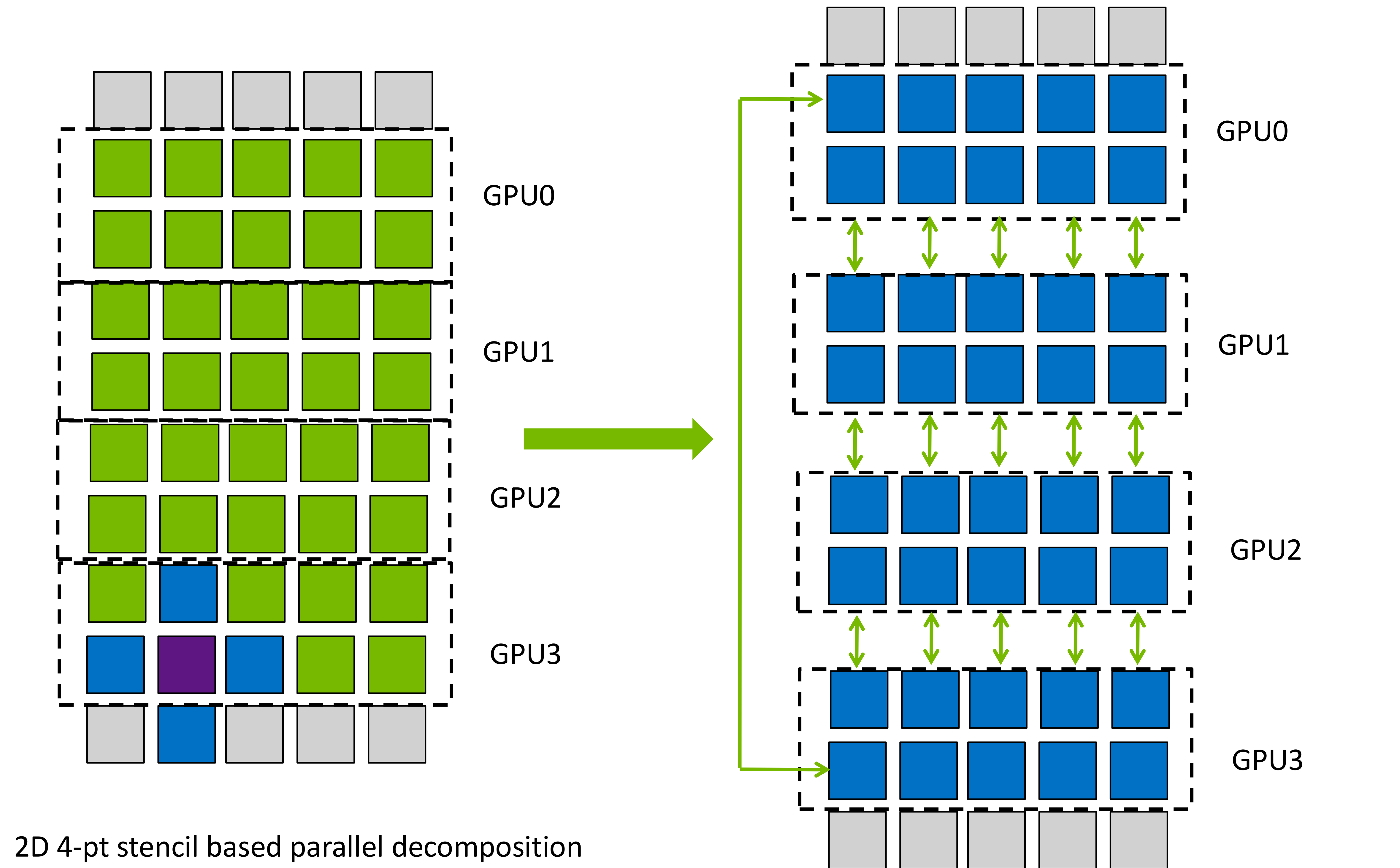
Device-initiated halo exchange

```
• jacobi<<<grid,block, 0,stream>>>
  ○ // Stencil Update
  ○ const real new_val =
    ■ 0.25 * (a[iy * nx + ix + 1] +
    ■ a[iy * nx + ix - 1] +
    ■ a[(iy + 1) * nx + ix] +
    ■ a[(iy - 1) * nx + ix]);
  ○ a_new[iy * nx + ix] = new_val;
```

Halo exchange

nvshmem_float_p

nvshmemx_float_put_nbi_block



Jacobi solver

Overlapping computation and communication

GPUs support multiple CUDA streams to run concurrently

So far, no overlap of communication and computation

Make sure that communication streams are scheduled

CUDA high priority streams!

```
int leastPriority = 0;
int greatestPriority = leastPriority;
cudaDeviceGetStreamPriorityRange(&leastPriority, &greatestPriority);

cudaStream_t compute_stream;
cudaStream_t push_stream;

cudaStreamCreateWithPriority(&compute_stream, cudaStreamDefault, leastPriority);
cudaStreamCreateWithPriority(&push_stream, cudaStreamDefault, greatestPriority);
```

Getting the range of
priorities

Assigning priority

Jacobi solver - Exercise 8a

NVSHMEM device-initiated halo exchange

Starter: [jacobi.cu](#) Reference: [jacobi_SOLVED.cu](#)

[08-applications/01-jacobi-solver/](#)

Implement global, block, and optional neighbor-only halo handoffs.

BASELINE

```
nvshmem_float_p(remote_ghost, value, neighbor)
nvshmemx_barrier_all_on_stream(compute_stream)
```

Push halo values; every PE joins before the next iteration.

BLOCK

```
nvshmemx_float_put_nbi_block(dest, src, n, pe)
```

All threads in the block participate in a halo-row chunk.

NEIGHBOR

```
nvshmemx_signal_op(...)
nvshmem_uint64_wait_until_all(...)
```

Signal and wait only for top/bottom neighbors before the next iteration.

Jacobi with NVSHMEM

Reference: device-initiated halo exchange

The reference issues device puts from the Jacobi kernel and overlaps interior computation with halo traffic.

Compare `jacobi.cu` with `jacobi_SOLVED.cu`.

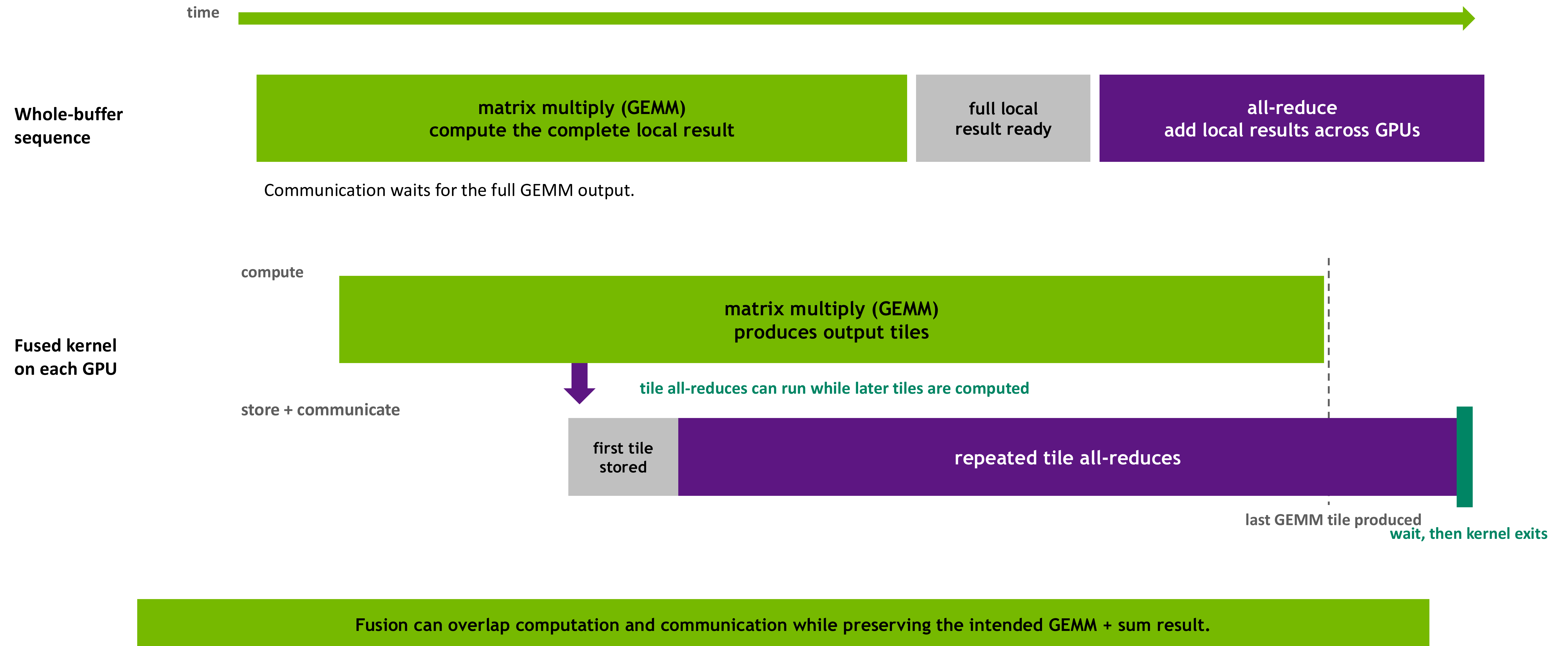
Then try `-use_block_comm`, `-neighborhood_sync`, and `-norm_overlap`.



Fused GEMM + all-reduce

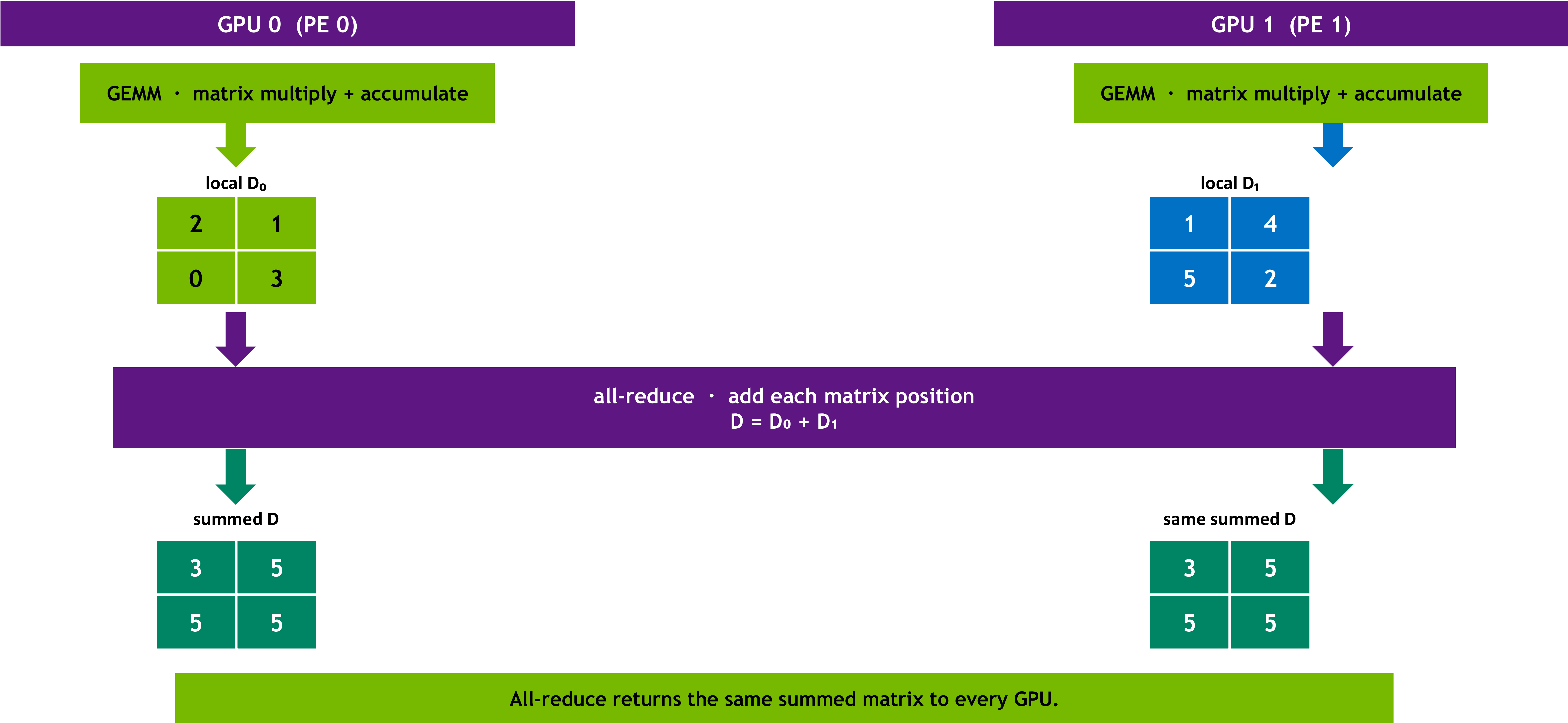
Why fuse GEMM + all-reduce?

In a conventional whole-buffer sequence, all-reduce starts after GEMM finishes



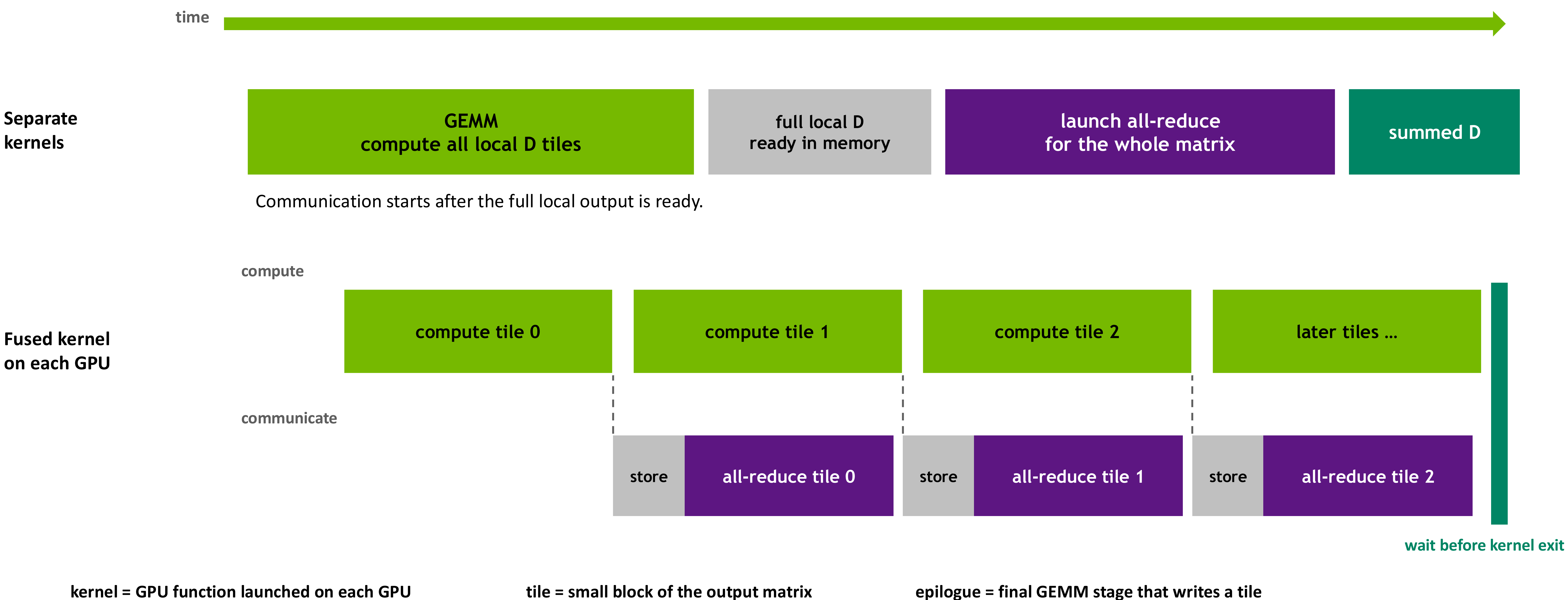
Why this calculation uses all-reduce

The required output is the elementwise sum of each GPU's local GEMM result



Fusion starts communication tile by tile

The kernel can all-reduce a completed tile while computing later tiles



The epilogue stores a tile; this fused kernel then starts its all-reduce while later tiles compute.

Fused GEMM + all-reduce - Exercise 8b

Tile-granular NVSHMEM collective in a CUTLASS epilogue

Starter: [gemmAR_fusion_blackwell_fp16.cu](#)

Fill in the two TODOs:

1. Define the tile-granular all-reduce collective.
2. Populate its arguments from symmetric buffers, rank/world size, and per-block teams.

Compare with `gemmAR_fusion_blackwell_fp16_SOLVED.cu`.

```
using CollectiveAllReduce =  
    CollectiveAllReduceMulticastWarpSpecialized<...>;  
  
arguments.allReduceArgs = {  
    block_D.get(), block_D_red.get(), stride_D,  
    nvshmem_my_pe(), nvshmem_n_pes(), teams_dev  
};
```

Each GEMM CTA uses a device-visible NVSHMEM team to reduce its output tile inside the epilogue. All participating blocks must be resident.

References

Blogs, Documentation, Guides

- NVSHMEM Docs
 - [API Documentation](#)
 - [Quickstart](#)
 - [Enhancing Application Portability and Compatibility across New Platforms Using NVIDIA Magnum IO NVSHMEM 3.0](#)
 - [Improving Network Performance of HPC Systems Using NVIDIA Magnum IO NVSHMEM and GPUDirect Async](#)
- NCCL Docs
 - [API Documentation](#)
 - [New Scaling Algorithm and Initialization with NVIDIA Collective Communications Library 2.23](#)
 - [Understanding NCCL Tuning to Accelerate GPU-to-GPU Communication](#)
 - [Enabling Fast Inference and Resilient Training with NCCL 2.27](#)
 - [NCCL Deep Dive: Cross Data Center Communication and Network Topology Awareness](#)
- GPU Mode Talk on Youtube: <https://www.youtube.com/live/2xMzQ1Z2Qe0?feature=shared>



NCCL Github



NVSHMEM Dev
forum